O'REILLY®

Head First Swift

A Learner's Guide to Programming with Swift

Early Release
RAW & UNEDITED

Paris Buttfield-Addison
& Jonathon Manning

A Brain-Friendly Guide

Head First Swift

A Learner's Guide to Programming with Swift

Paris Buttfield-Addison and Jon Manning

Head First Swift

by Paris Buttfield-Addison and Jon Manning

Printed in the United States of America.

Published by O'Reilly Media, Inc. , 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (http://oreilly.com). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com .

Editors: Michele Cronin and Suzanne McQuade

Production Editor: Kristen Brown

Copyeditor: FILL IN COPYEDITOR

Proofreader: FILL IN PROOFREADER

Indexer: FILL IN INDEXER

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

December 2021: First Edition

Revision History for the First Edition

- 2021-03-05: First Early Release

See http://oreilly.com/catalog/errata.csp?isbn=9781491922859 for release details.

978-1-491-92285-9

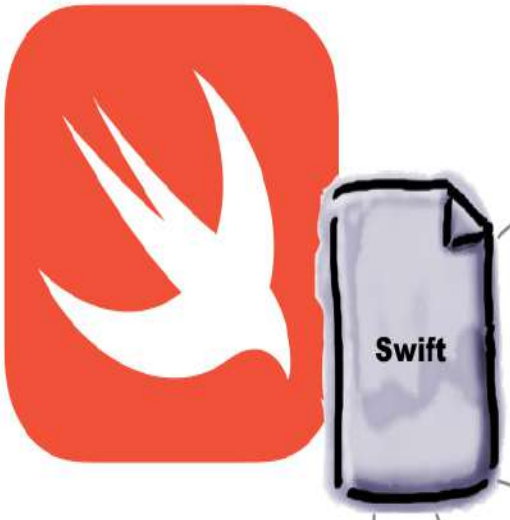
[FILL IN]

Chapter 1. Introducing Swift: Apps, web, AI, and beyond!

Swift is a programming language you can rely on. It's a programming language that you can take home to meet your family. Safe, reliable, speedy, friendly, easy to talk to. And, while Swift is

most well known for being the programming language of choice for Apple's platforms (such as iOS, macOS, watchOS, and tvOS), the open source Swift project also runs on Linux, and is gaining ground is a **systems programming** language, as well as on the server. Swift for Windows is even in the works. You can build everything from mobile apps, to games, to web apps, to frameworks and beyond. Let's get started!

Swift is a language for everything



macOS , Linux, and Windows software can be created with Swift. On macOS, you can make rich graphic applications, using a number of user interface frameworks, which we'll get back to later.

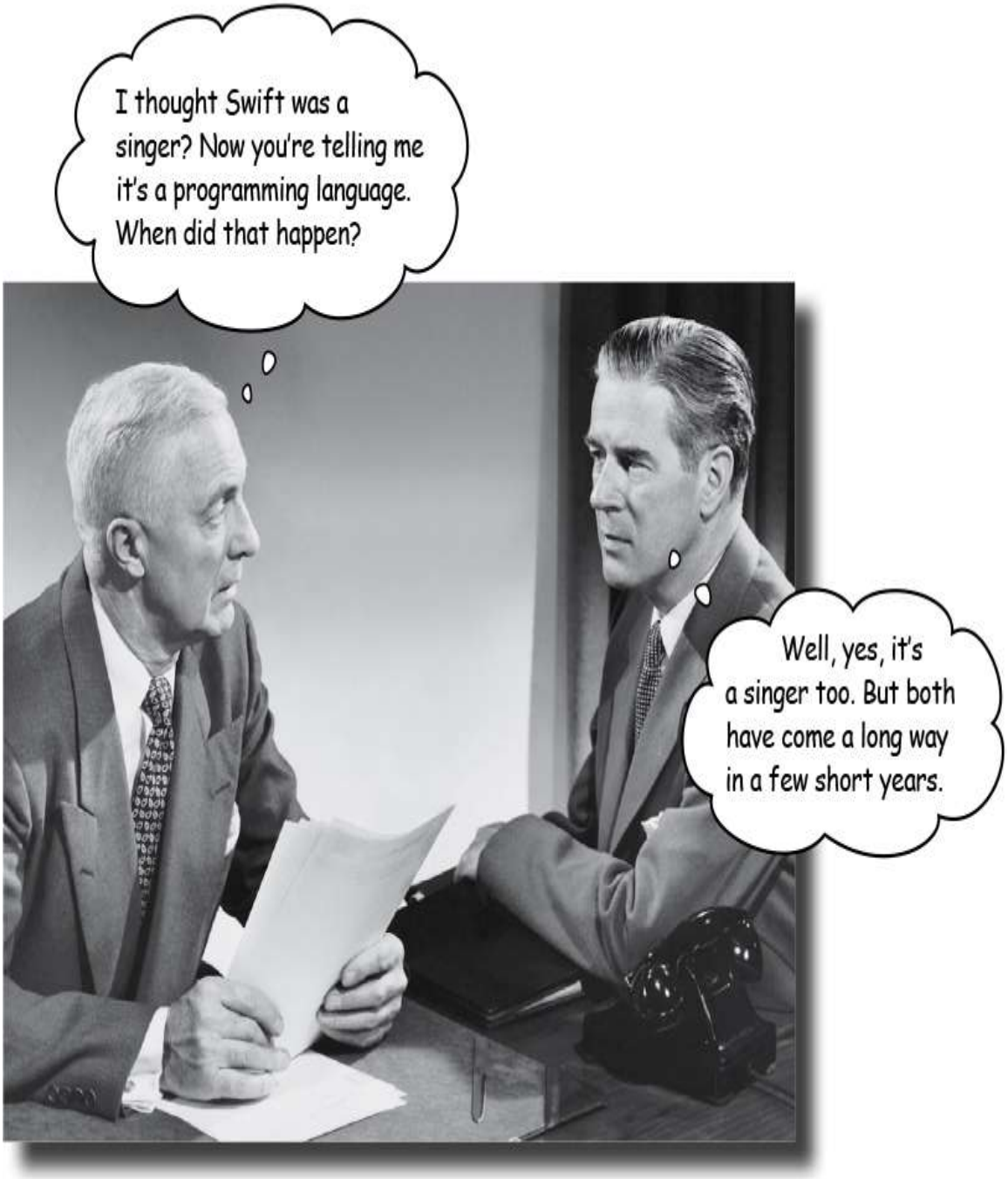
iOS and iPadOS apps can be created with Swift, for iPads, iPhone, iPods touch, as well as apps for tvOS, which powers Apple TV, and watchOS, which powers Apple Watch.

Data science and **machine learning** can be performed with Swift, as well as **systems programming**, for architecting all kinds of science, simulations, and beyond.

Swift is also Open Source. The Swift Open Source project is vibrant, well-managed, and has a friendly and clear process for getting involved.

We'll discuss how you can get involved later in the book.

Web applications can be created with Swift, driving the backend of websites, or statically generating pages for your website.



Swift has evolved fast.

From humble begins with Swift 1, all the way through to today, with Swift 5.x. Each major Swift update has added new language

features, as well as deprecating certain syntax elements. Since its announcement and release in 2014, Swift has been one of the fastest growing, and most popular, languages in programming language history.

Swift has been placed in the top 10 programming languages for the past few years, and continues to grow in popularity, amount of projects using Swift, and userbase. Swift skills are also in high-demand, so don’t forget to add it your résumé when you’re finished with this book.

Swift originally started as a language solely for Apple’s platforms—iOS, macOS, tvOS, watchOS—but has grown and expanded its possibilities since it was open sourced in 2015.

The swift evolution of Swift

2014

Swift 1

Swift was introduced, to great surprise and fanfare, at Apple's big developer conference (WWDC) in June 2014. At this point, Swift was a proprietary language, only useful for developing for Apple's platforms, and it was closed source.

The first few releases of Swift only supported Xcode on macOS.

2015

Swift was open sourced in December 2015, at version 2.2. Since then, Swift's development takes places in the open at https://github.com/apple/swift

Swift 2

Swift 2 arrived on the scene at WWDC 2015, changing a lot of the syntax based on community feedback, and envolving the features of the language. Swift continued to grow in popularity and community acceptance.

2016

Swift 3

Swift 3 released in September 2016, and brought a bigger raft of syntax changes, again based on community feedback. Swift 3 heralded a new era of stability for Swift, with the promise that the syntax would change less in the future.

Swift Playgrounds was released for the iPad at the same time Swift 3 released..

2017

Swift 4

Swift 4 came out in September 2017, bringing new features, and maintaining slightly more stability between versions than previously. The Swift Open Source project continued to gain ground, and the popularity continued to skyrocket.

Swift was designed to take the best elements of other languages, including Objective-C, Rust, Haskell, Ruby, Python, C#, and many others.

2018

SwiftUI, the Swift-powered user interface framework was launched in mid-2019.

2019

Swift 5

Swift 5, the major version you're learning in this book, was released in March 2019, and introduced a full stable version of Swift to the world. Right now, Swift is continuing to rise in popularity, and features. It's a great time to learn.

2020

Swift Playgrounds was released as a macOS application in February 2020.

SOMETHING ABOUT THE FUTURE OF SWIFT

How you're going to write Swift

Every programming language needs a tool for you to program in and use to run the code you're writing. Basically, you can't do much with Swift without running some Swift code.

So, how's that done? We'll be using an app developed by Apple called **Playgrounds**!

Playgrounds are just like (collections of) big old-fashioned text files, where it also turns out you can run each and every line written through the Swift compiler, and see the result.

Much of the Swift you'll be writing via this book will involve Playgrounds.

> **NOTE**
>
> But not all of it. We'll explain more, a bit later.

Working with Swift in Playgrounds involves three stages:

Writing

1

You write your Swift code in a Playground, which is a **programming editor** just like you might find when working with a variety of other languages. You can put all your Swift code in one file, or you can split it up across *multiple files*. It's your choice.

We'll talk about how you can do this a bit later.

Running

2

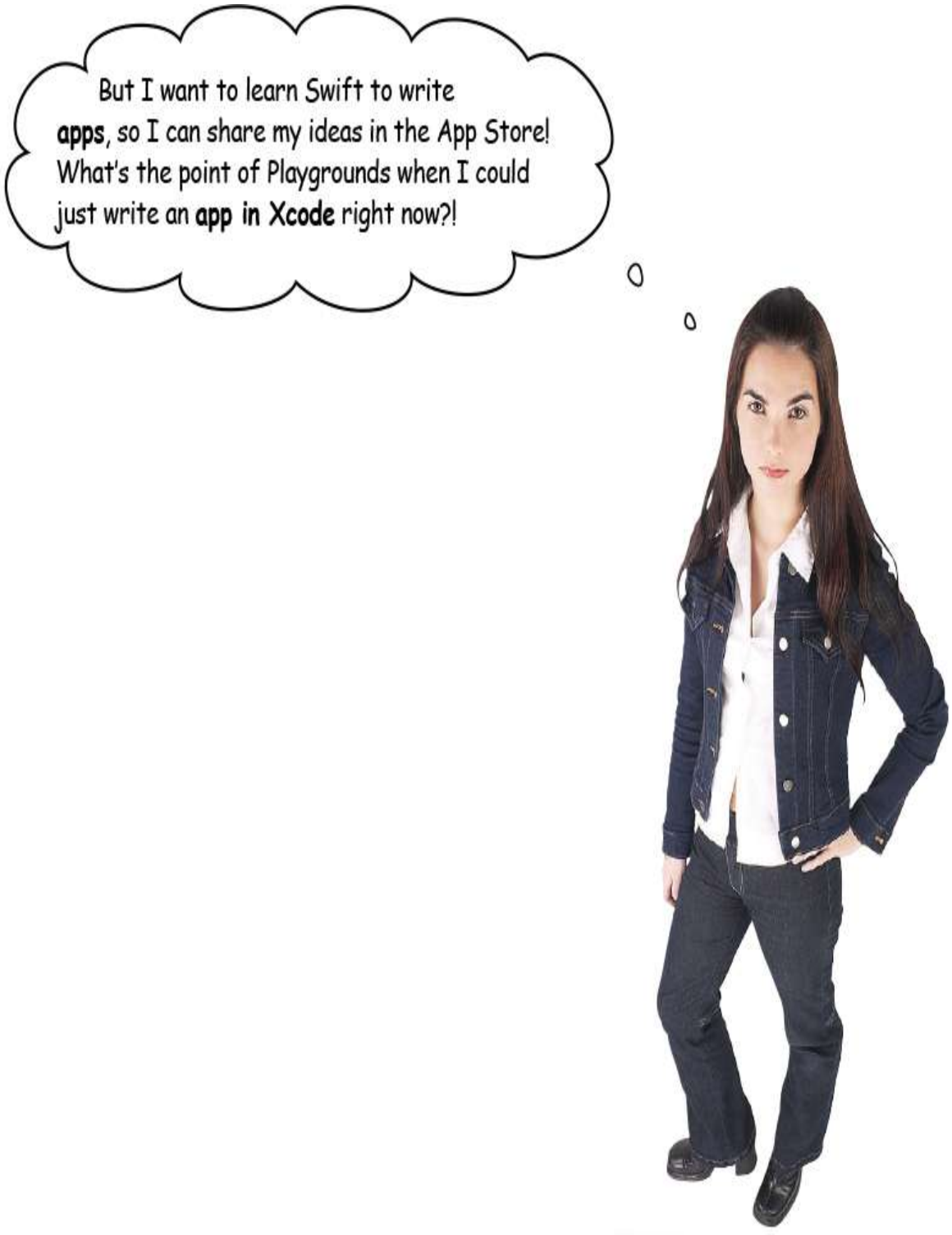
You can then run your Swift code, either *all at once*, or *line-by-line*, within the Playground. You can see errors, as well as the *output* of your code, inline next to each line, as well as in the console below the code editing area.

Using a Playground is the fastest way to switch between running and tweaking, so you can get a good understanding of what's going on.

Tweaking

3

The Playground has such a rapid write-run cycle that it's easy to make tweaks, improve your code, and evolve it into a form you need to accomplish your programming goals. Swift's speed of development is one of its primary selling points. Use it wisely!



It's easier, and better, to learn Swift in a Playground.

By working in a Playground now, you don't have to worry about all the *boilerplate* code that comes with building an app, as well as

compiling and running your code into an app, testing it on an iOS device simulator, testing it on a physical iOS device, and troubleshooting your code for the slight differences between all the various pieces of iOS and iPadOS hardware.

A Playground lets you *focus on the Swift code*, and not worry about any of the other things that come with building an app (or writing a website backend, or doing data science, or any of the other things you might want to do with Swift). That's why we start in Playgrounds.

It removes all the distractions.

It's not lesser in any way, and it's real, actual Swift code, and lets you work up to building **apps** and more, using Swift.

> **NOTE**
>
> If you're here because you want to write iOS apps (that's awesome, by the way!) then please stick with us through the Playgrounds. It really is the best way to learn.

Xcode vs Playgrounds

It's really tempting to jump straight to Xcode when you start learning Swift, particularly if you're already a programmer.

If you want to learn Swift instead of learning Swift plus the idiosyncracies of the (very powerful, very complex) Xcode environment, then start with Playgrounds. It lets you focus on the Swift.

The path you'll be taking

Playgrounds

Playgrounds are our starting point. They're available for macOS and for iPad OS, and they let you code real Swift, using the Swift compiler, and quickly view the output, test new ideas, and separate your work into logical pages within a single playground. We cannot emphasis this enough: everything you do in a Playground is real Swift.

iOS, macOS, tvOS, and watchOS apps

Once our Swift skills are big and strong, we'll take them out of the Playground and into Xcode. In Xcode, we'll learn to use Swift to make apps for Apple's platforms.

Along the way, we'll learn SwiftUI, Apple's Swift-powered user interface framework. We'll start using it in Playgrounds, and move to Xcode a little later.

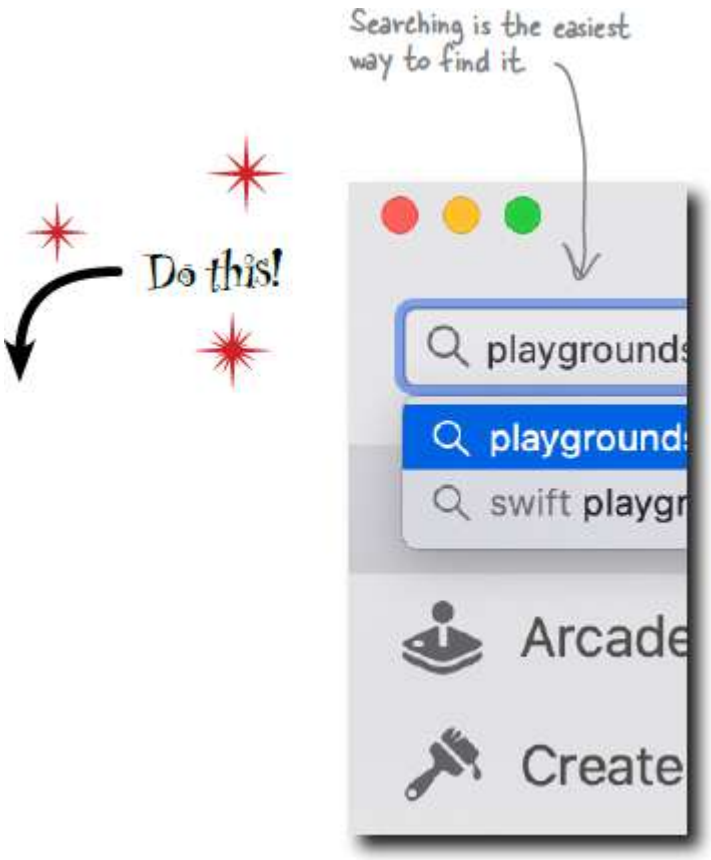
Websites

And, to cap things off, we'll take Swift into the web, with Vapor and Publish, frameworks for building websites and backends using Swift, and Swift for TensorFlow, a powerful scientific computing and machine learning framework backed by Google.

Data Science

Getting Playgrounds

Swift Playgrounds is an app that's available for macOS and iPad OS. You use the Playgrounds app to write, run, and tweak your Swift code.



Downloading and installing Playgrounds on macOS

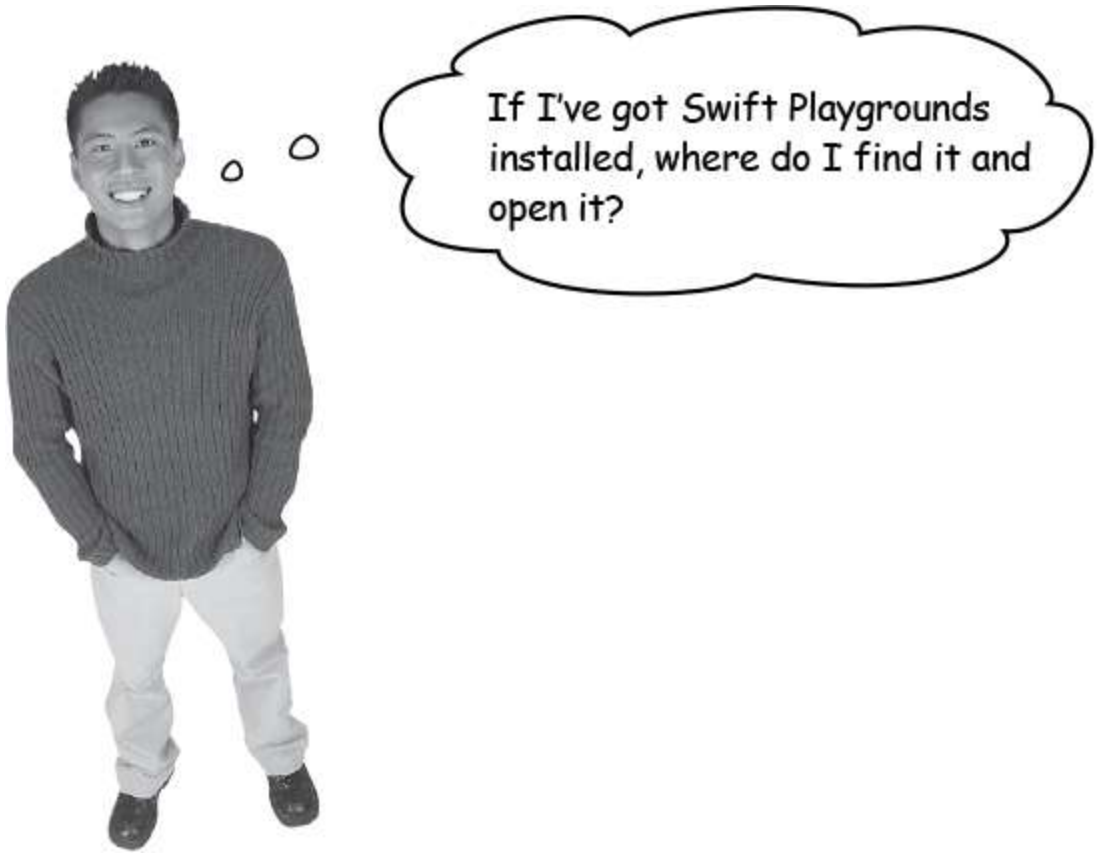
1. Open the App Store app on your computer, find the **search field**, select it, and enter "Playgrounds". Press return on your keyboard.

2. Wait for the App Store to load the search results page, and then click on the **Swift Playgrounds** entry in the list (the icon has a blueprint and a hammer.)
3. Click the **install button** and wait for Xcode to download and install on your computer.

WATCH IT!

It might be tempting to jump straight to Xcode.

Feel free to install Xcode, but we won't need it right now. We'll get back to it later in the book..



To launch Swift Playgrounds, browse to your Applications folder, and double click on the Swift Playgrounds icon.

If you prefer to use Launchpad or Spotlight to launch your other apps, you can do that with Swift Playgrounds too.

You can launch Playgrounds via Spotlight. Or just look for it in your applications folder

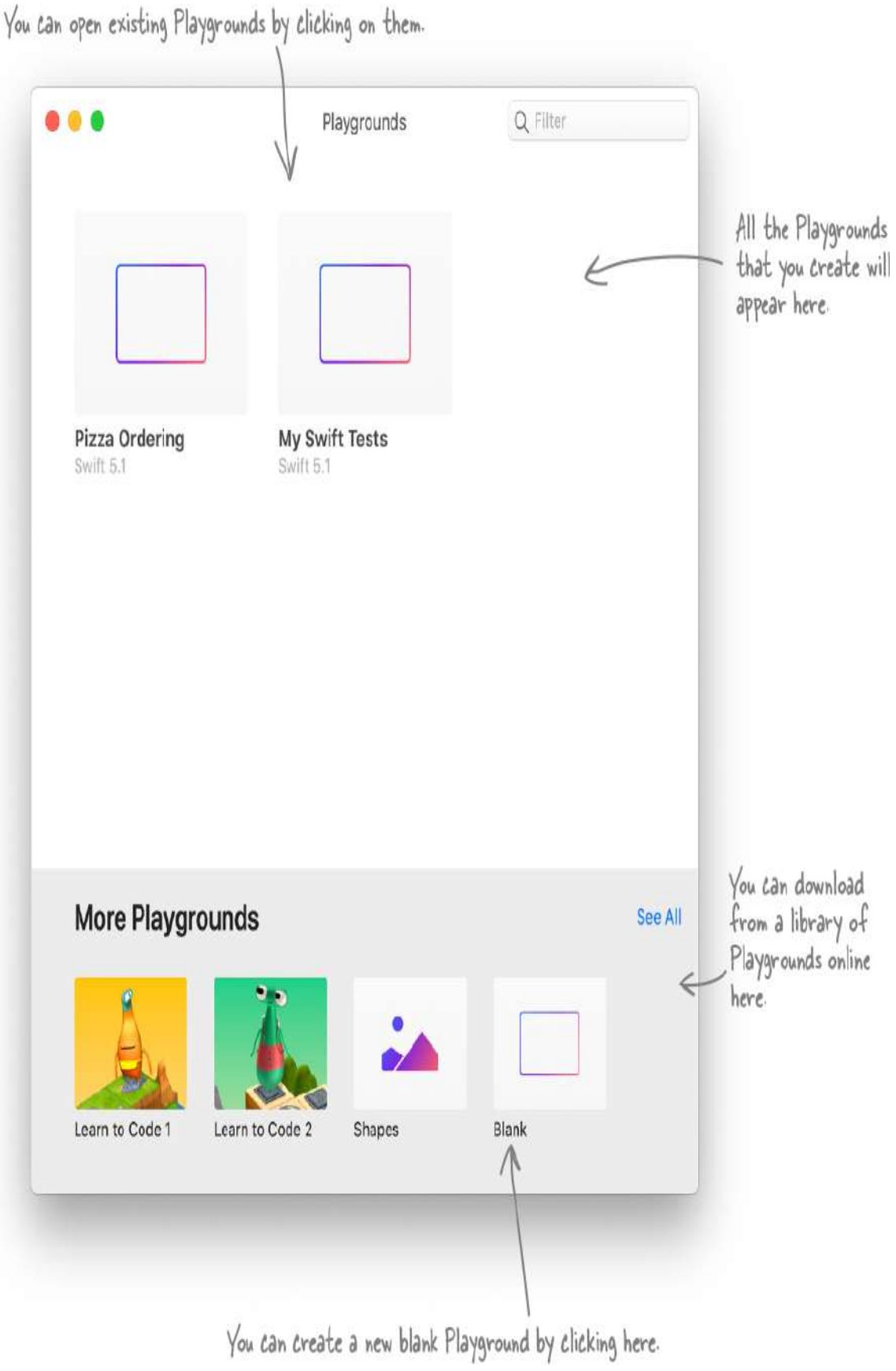
Or look for Playgrounds in your Applications folder.

BULLET POINTS

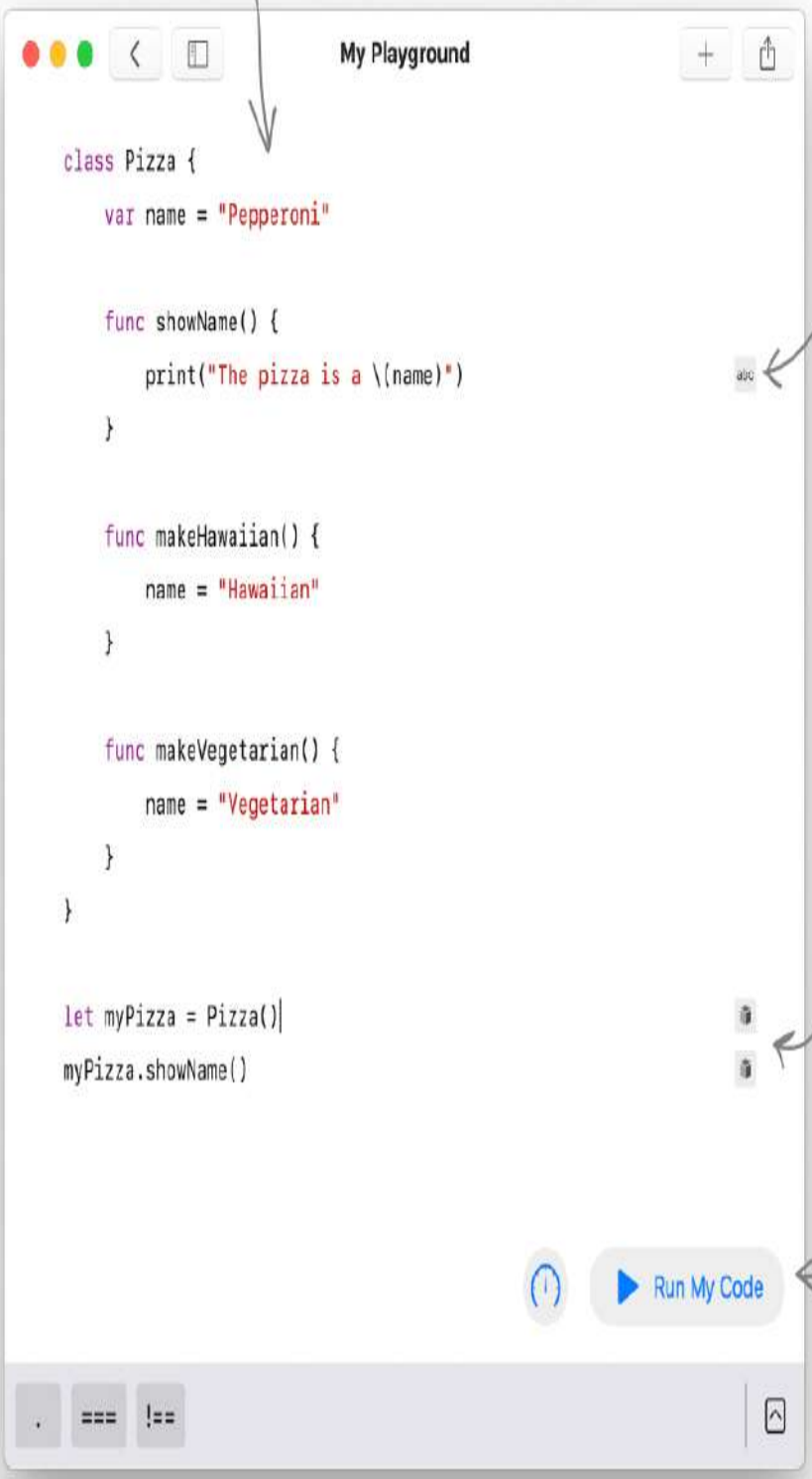
- Writing Swift requires a tool, or multiple different tools, that let you write Swift code, run Swift code, and tweak Swift code (before then running it again, and tweaking it again, and so on).
- One such tool is Apple's Playgrounds app, which is available for iPad OS and macOS. It's minimal, and keeps the focus on using the Swift programming language instead of having to learn the tool and the programing language at the same time.
- Another, more complicated tool, is Apple's Xcode. Xcode is a full integrated development environment (IDE), similar to Visual Studio and others. It's a very big, very complicated tool that serves many purposes.
- Xcode is necessary if you want to write apps for Apple's platforms, like iOS, and we'll be using Xcode later in the book.
- For learning Swift, the best tool is Playgrounds.

Creating a Playground

Once you've launched Swift Playgrounds, you can create a blank playground to work with, or open an existing Playground that you've previously created:


You can open existing Playgrounds by clicking on them.
Playgrounds
Filter
Pizza Ordering
Swift 5.1
My Swift Tests
Swift 5.1
All the Playgrounds that you create will appear here.
More Playgrounds
See All
You can download from a library of Playgrounds online here.
Learn to Code 1
Learn to Code 2
Shapes
Blank
You can create a new blank Playground by clicking here.

A Playground will highlight your code in pleasant colours that make iteasier for you to read.



If a piece of Swift code has a result, you'll see a small icon appear next to the line.

You can click a result icon to create a Viewer, to display the output or result of a particular line.

You can run your code using this button.

A SWIFT TEST DRIVE 

Go ahead and type the contents of the Playground shown on this page into a brand new Swift Playground of your own.

Run it. What does it display? Use the result icons to add viewers where there are results to view.

We haven't covered all the features of Swift used in this code yet, but can you figure out how to change the pizza created to a different kind?

Using a Playground to code Swift

1 Create a Playground

Create a Playground in the Playgrounds app

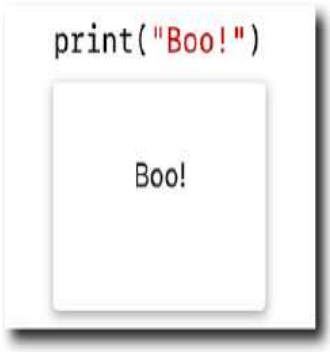
2 Write some code, then run it

Write your code—as little or as much as you need— then run the code by hitting the button.

The run button might get smaller if you use a small Playgrounds window.

3 Look at the outputs

Once your code has run, lines where there's an output display an indicator which you can click to add a viewer.



Basic building blocks

Our journey through Swift will take a while, but we can get up to speed on the basics pretty fast. We're going to speed through the basics of Swift to give you a taster of many areas of the language.

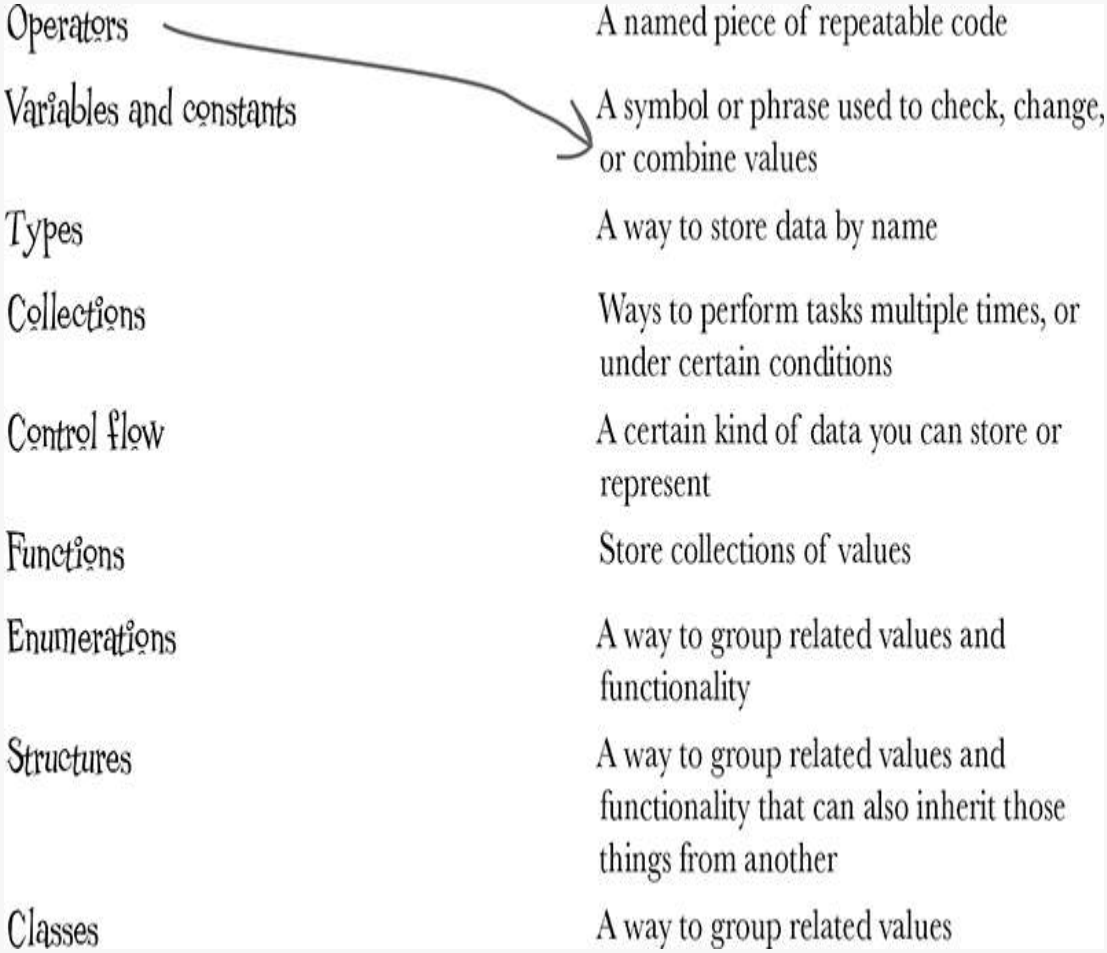
After this, we'll come back and take a chapter-by-chapter approach to unpack each and every feature of Swift.

There are certain basic building blocks that make up almost every programming language on the planet (and probably any programming languages that might exist off the planet, too; programming is pretty universal).

Those basic building blocks consist of things like **operators**, **variables and constants**, **types**, **collections, control flow, functions, enumerations, structures**, and **classes**.

WHO DOES WHAT?

We haven't told you what each basic building block does yet, but you might be able to guess. Match each building block to what it does. The solution is over a few pages, too (and we've done one, to start you off).

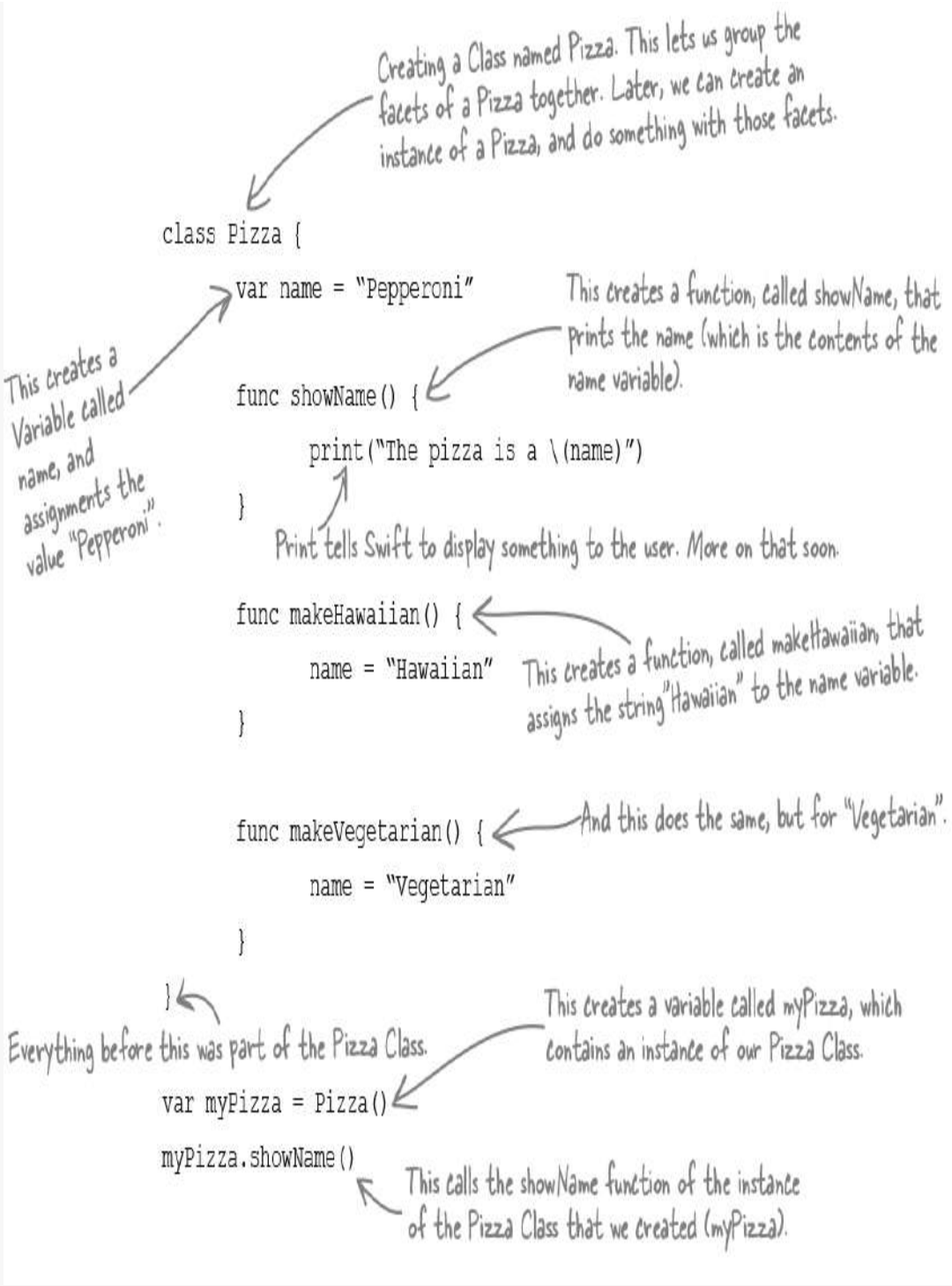


RELAX

Just relax. We don’t expect you to be reading Swift code, memorising Swift’s history and evolution, and spouting off facts about Swift like you invented it.

However, you spent your hard-earned pennies on this book, so we’re not going to let you get away without jamming the tender morsels of Swift knowledge into your mind. You might not understand everything yet, but in time (swiftly, you might say) you will recognise how the building blocks fit together.

Remember the code a few pages ago? We’re going to go through it to get an idea of what it might do.


Creating a Class named Pizza. This lets us group the facets of a Pizza together. Later, we can create an instance of a Pizza, and do something with those facets.
class Pizza {
var name = "Pepperoni"
This creates a Variable called name, and assignments the value "Pepperoni".
This creates a function, called showName, that prints the name (which is the contents of the name variable).
func showName() {
print("The pizza is a \(name)")
}
Print tells Swift to display something to the user. More on that soon.
func makeHawaiian() {
name = "Hawaiian"
}
This creates a function, called makeHawaiian, that assigns the string "Hawaiian" to the name variable.
func makeVegetarian() {
And this does the same, but for "Vegetarian".
name = "Vegetarian"
}
}
Everything before this was part of the Pizza Class.
This creates a variable called myPizza, which contains an instance of our Pizza Class.
var myPizza = Pizza()
myPizza.showName()
This calls the showName function of the instance of the Pizza Class that we created (myPizza).

You don’t need a main method, or anything else (like semicolons).

In Swift, you can just start your code however you like. In the case of the Playgrounds we're working with at the moment, execution starts at the top of the file and proceeds from top to bottom.

Some of the building blocks that we'll be learning about, like Classes, won't be executed in a way that lets you see them working immediately, because they rely on have an instance of them created elsewhere in the code. We'll get to that though.

In general, though, you don't need main method, or any specific starting point.

When we move on from Playgrounds to building apps using Swift, we'll look at the various potential starting points an app can have. For now, though, push it from your mind.

You also don't need semicolons at the end of your lines, and whitespace has absolutely no effect on anything.

> **NOTE**
>
> If you're coming from other languages, you might be used to ending lines with a semicolon (;) or whitespace having some substantive meaning to your code's logic. None of these things matter, or apply, in Swift.

SHARPEN YOUR PENCIL

You've only just started learning Swift, but you're a clever egg, and can probably make a pretty good guess about what's going on in some Swift code. Take a look through each line of code in the Swift program below, and see if you can guess what it does. Write your answers below. There's one that's already completed to get your started! If you want to check your answers, or if you get stuck, you can find a completed version on the next page.

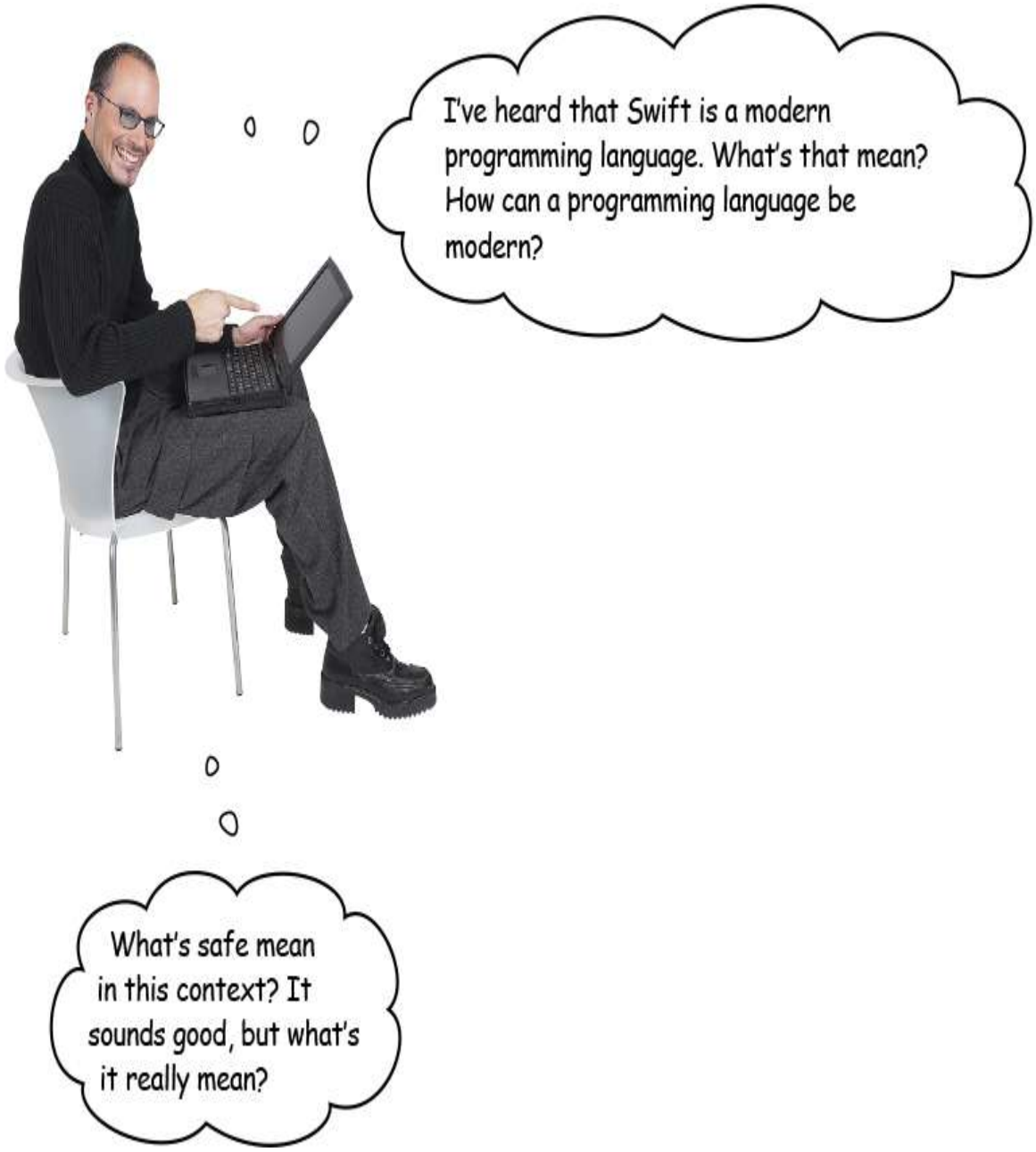
```
class Message {
    var message = "Message is: 'Hello from Swift!'"
    var timesDisplayed = 0

    func display() {
        print(message)
        timesDisplayed += 1
    }

    func setMessage(to newMessage: String) {
        message = "Message is: '\(newMessage)'"
        timesDisplayed = 0
    }

    func reset() {
        timesDisplayed = 0
    }
}

let msg = Message()
msg.display()
msg.timesDisplayed
msg.display()
msg.timesDisplayed
msg.setMessage(to: "Swift is the future!")
msg.display()
msg.timesDisplayed
```



Swift the culmination of decades of programming language development and research.

It's modern because it has incorporated the learnings of other programming languages, and refined the user experience— the user being the programming—to the point where it's easier to read the code and easier to maintain the code. There's **less to type**,

compared with many languages, and Swift's language features make the **code cleaner** and **less likely to have errors**. Swift is a safe language.

Swift also supports features that other languages often do not, such as **international languages**, **emoji**, **Unicode** and **UTF-8 text encoding**. You also don't need to pay much attention to memory management. It's kind of magic.

Swift is safe because it automatically does many things that other languages require code for, or make it challenging to do at all.

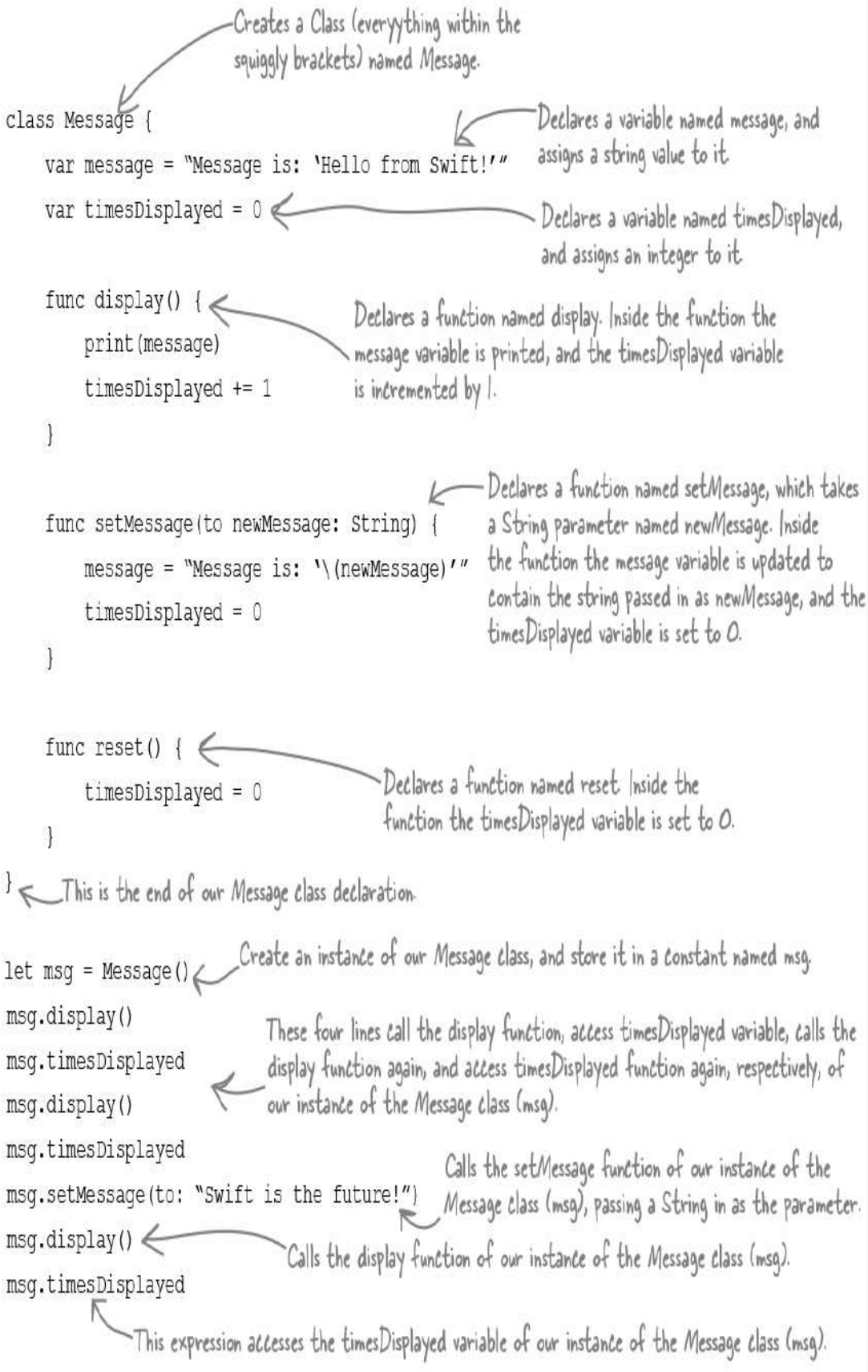
There's a lot of reasons why Swift is often described as "safe". For example, variables are always initialized before use, and arrays are automtically checked for overflow, and the memory your program uses is automatically managed.

Swift also heavily leans on the use of value types, which are types that are copied instead of referenced. so you know it won't be modified anywhere else when you're in the midst of using it for something.

Swift's objects can never be nil, which helps you prevent runtime crashes in your programs. You can still use nil though, as and when you need it, via optionals.

SHARPEN YOUR PENCIL SOLUTION

Don’t worry about whether you understand any of this yet! Everything here is explained in great detail in the book, most within the first 40 pages). If Swift resembles a language you’ve used in the past, some of this will be simple. If not, don’t worry about it. We’ll get there....


Creates a Class (everyything within the squiggly brackets) named Message.
class Message {
Declares a variable named message, and assigns a string value to it.
var message = "Message is: 'Hello from Swift!'"
var timesDisplayed = 0
Declares a variable named timesDisplayed, and assigns an integer to it.
func display() {
print(message)
timesDisplayed += 1
}
Declares a function named display. Inside the function the message variable is printed, and the timesDisplayed variable is incremented by 1.
Declares a function named setMessage, which takes a String parameter named newMessage. Inside the function the message variable is updated to contain the string passed in as newMessage, and the timesDisplayed variable is set to 0.
func setMessage(to newMessage: String) {
message = "Message is: '\(newMessage)'"
timesDisplayed = 0
}
func reset() {
timesDisplayed = 0
}
Declares a function named reset. Inside the function the timesDisplayed variable is set to 0.
}
This is the end of our Message class declaration.
Create an instance of our Message class, and store it in a constant named msg.
let msg = Message()
msg.display()
msg.timesDisplayed
msg.display()
msg.timesDisplayed
These four lines call the display function, access timesDisplayed variable, calls the display function again, and access timesDisplayed function again, respectively, of our instance of the Message class (msg).
Calls the setMessage function of our instance of the Message class (msg), passing a String in as the parameter.
msg.setMessage(to: "Swift is the future!")
msg.display()
Calls the display function of our instance of the Message class (msg).
msg.timesDisplayed
This expression accesses the timesDisplayed variable of our instance of the Message class (msg).

A Swift Example

All this Swift is hard work. A pizza is required! It just so happens that the local pizza place heard you were learning Swift, and wants to put your new skills to work.


We need some new pizza names. Can you code something up to generate some names for me? I'm not very creative.
The pizza chef has confidence in your Swift skills.
Just some ideas so far.
Pizza name ideas
Swiss Hawaiian
Cheesey Meat
Bacon and Avocado
Apple and Gruyere
Tomato and Tomato

Here's what you need to build

You're going to make a quick, little Swift Playground that takes the list of all possible pizza ingredients the chef has available, and generates a random pizza with four of those ingredients. Got it?

There's three things you'll need to do to make this happen:

1. **A list of ingredients**

   You'll need a list of ingredients. The chef can help with the list, but we'll need to store it somehow. A `String Array` is probably the most sensible solution.

2. **A random choice of ingredient**

   You'll also need a way to pick a random ingredient from that list. Four times over (because our pizza needs four ingredients).

If we use an `Array` to store our ingredients list, then we can use the `randomElement()` function on our array to get back a random ingredient.

3. **Displaying the randomly generated pizza**

   Finally, you'll need to display the randomly generated pizza in the form of "ingredient 1, ingredient 2, ingredient 3, and ingredient 4".

The print function

You might have noticed that sometimes our code contains a line that looks like this:

```
print("Some text to get printed!")
```

This is Swift's **print function**. It tells Swift to display the given items as text. Sometimes you'll see it used like this, too:

```
print("The pizza is a \(name)")
```

Used like this, print will substitute the value of the variable wrapped in **\(** and **)** into what's printed. Handy, right? We'll cover this more later in the book, when we talk about **string interpolation**.

❶ Building a list of ingredients

The first step is to store the list of ingedients that the chef provided in a useful data structure.

We’re going to use one of the collection types that Swift provides: specifically, we’re going to make an Array of Strings. Each String stored in the Array will be one of the ingredients.

We’ll create our array of ingredients like this:

```
let ingredients = [
                  "Pepperoni", "Mozzarella",
                  "Bacon", "Sausage",
                  "Basil", "Garlic", "Onion", "Oregano",
                  "Mushroom", "Tomato",
                  "Red Pepper",
                  "Ham", "Chicken",
                  "Red Onion",
                  "Black Olives",
                  "Bell Pepper",
                  "Pineapple",
                  "Canadian Bacon", "Salami",
                  "Jalapeño",
                  "Spinach",
                  "Italian Sausage", "Provolone",
                  "Pesto", "Sun-Dried Tomato",
                  "Feta",
                  "Meatballs",
                  "Prosciutto",
                  "Cherry Tomato",
                  "Pulled Pork", "Chorizo",
                  "Anchovy", "Capers"
]
```

There are 33 things in this array of strings (which is storing ingredients). This means you can access item 0 to 32, because the first thing in a Swift array is item 0. We'll be using arrays more in the next chapter.

This is item 32.

EXERCISE

Create a new Swift Playground, and code your list of ingredients.

Use the print function to to print some of the ingredients. You can access an individual ingredient using this syntax: `ingredients[7]`. What ingredient is the 7th item in the array?

2 Picking four random ingredients

The second step is to pick a random ingredient four times over (from the list that we just put into an `Array`).

Swift's `Array` collection type has a handy function, `randomElement()`, which returns a randomly selected element from the array it is called on.

To get a random element from the ingredients Array, we can do this:

```
ingredients.randomElement()!
```

We need to put the ! at the end, because this function returns an ***Optional***. We'll come back to what that means in a chapter or two, but the summary is: there might be a value there (if there are things in the array), and there might not (because an array can be empty, yet still perfectly valid).

So, to be safe, Swift has the concept of Optionals, which may, or may not, contain a value. The ! forces the possibility of an optional to be ignored, and if there's a value it will just give us the value.

If there was any possibility of this `Array` changing (there's not, because it's been declared as a constant—which cannot ever change–using the let keyword) then this would be unsafe programming (because we'd be calling `randomElement()` and disregarding the possibility of an optional when, if it wasn't a constant, the `Array` has the possibility of being empty).

`randomElement` is a convenience property that we can use on our array (and other Swift collection types). There are other properties and conveniences on arrays, including first and last (which return the

`first` and `last` array element, respectively) and `append`, which adds something new to the end of the array.

> **NOTE**
>
> You'll learn all about Swift's collection types (which are ways to store collections of values) in the next chapter.

EXERCISE

Open your Swift Playground that contains the list of ingredients.

Use the print function again, and print a selection of random ingredients using `ingredients. randomElement()!`

Try printing the last element of the array, by using the `last()` property. What is it?

Append a new ingredient to the end of the array using the syntax `ingredients. append("Banana")`, then print the last element of the array again to check that it's got your new ingredient where you expected it to be.

3 Displaying our random pizza

Finally, we need to use our Array of ingredients, and the `randomElement()` function to generate and display our random pizza.

Because we want four random ingredients, we need to call this four times, so we may as well do it inside a call to the `print` function, like this:

```
print("\(ingredients.randomElement()!), \
(ingredients.randomElement()!),
\(ingredients.randomElement()!), and \(ingredients.randomElement()!)")
```

Yes, you might get the same random ingredient more than once.

Each call to `randomElement()` has no knowledge of how you're using its returned element, or that you're calling it multiple times. So yes, it is possible you might generate a pizza that's nothing but Pineapple.

> **NOTE**
>
> ```
> Red Pepper, Ham, Onion, and Pulled Pork
> Pineapple, Mozarella, Bacon, and Capers
> Salami, Tomato, Prosciutto, and Meatballs
> Pineapple, Pineapple, Pineapple, and Anchovy
> ```

EXERCISE

Comment out everything but the list of ingredients array in your Swift Playground.

Implement the code that's needed to print a four ingredient randomly generated pizza, as above.

Test it out. Does it work OK?

The chef seems to think that it might be annoying to get the same ingredient more than once

Can you figure out how to code it so that that never happens? Give it a go.

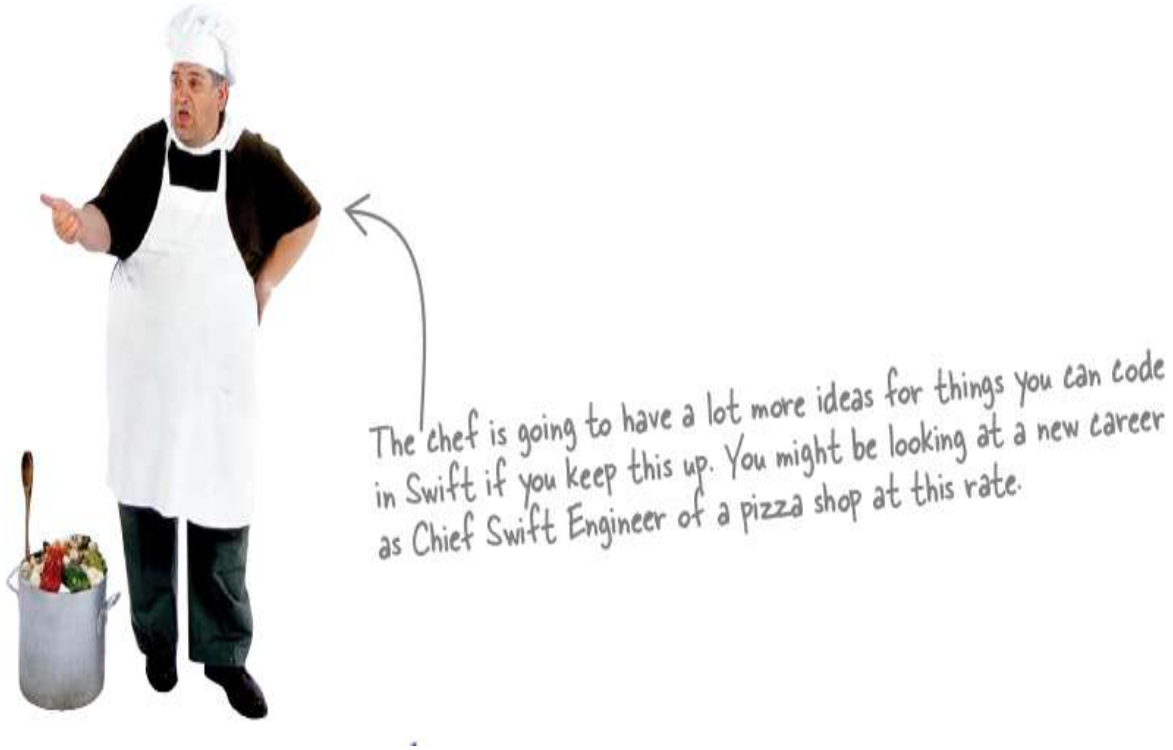
Congrats on your first Swift steps

You've done a lot in this chaper. You've been dropped in the deep end, and been asked to decipher some actual, real Swift code as you became familiar with programming Swift in a Playground.

You've also built a little tool to help the pizza chef generate random names for his pizzas. Very useful!

In the next chapter, we're going to write some longer-form Swift code. Some real, serious, actual code that does something useful (kind of). You'll be learning all the Swift building blocks: **operators**, **variables and constants**, **types**, **collections, control flow, functions, enumerations, structures**, and **classes**.

Before you get there, we've got a few questions to answer that you might have, a crossword puzzle, and a tiny bit more sassy commentary. And don't forget to get a nice cup of coffee, or some sleep.



There are no Dumb Questions

Q: Where are the semicolons? I thought programming was meant to be full of semicolons!

A: Swift doesn't end lines with semicolons. If it makes you feel more comfortable, you can do it anyway. It's still valid, it's just not necessary!.

Q: How do I run my code on an iPhone or iPad? I bought this book so I could learn to make iOS apps and get filthy rich.

A: We'll explain everything you need to know to use your Swift knowledge to make iOS apps much, much later in the book. We make no promises about getting rich, but we do promise that you'll know how to build iOS apps by the end. Again, be patient.

Q: I'm used to Python. Does whitespace mean anything in Swift?

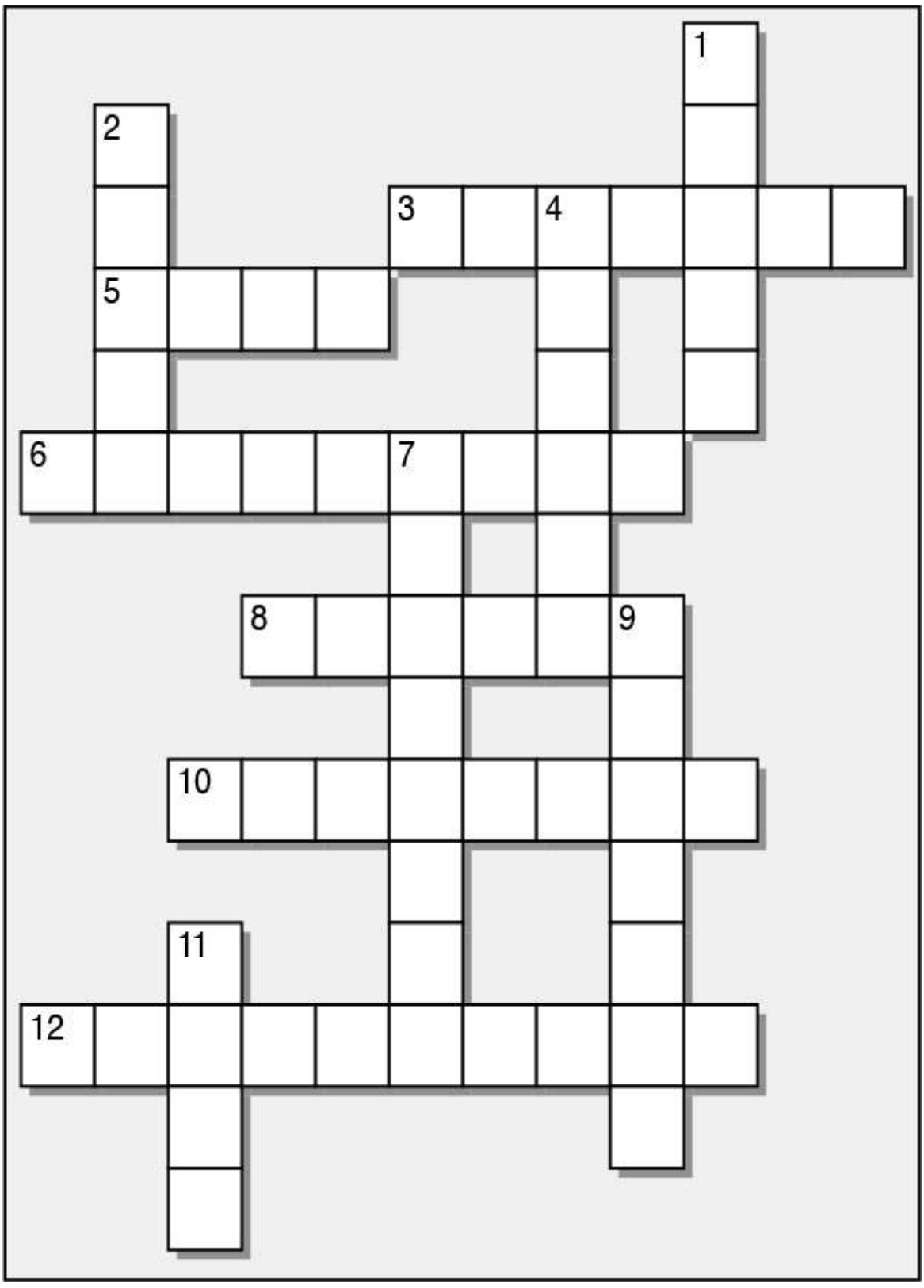
A: No, whitespace is not meaningful in Swift the same way it is in Python. Don't worry about it!

Q: I still don't really understand why everyone keeps going on about Swift being "safe"? I don't understand how a programming language can be safe.

A: Safe is just a shorthand way of saying "it's harder to make mistakes with". Swift does everything it can to help you catch mistakes as early as possible. If it can throw an error when you're compiling a program, instead of when it's running (potentially causing a crash) then it will. Swift's "safety" just means it tries its hardest to stop you writing code that could crash.

Swiftcross

Put your brain to the test with some Swift terminology. Answer are over the page. Check back through the chapter if you get stuck!



Across

3) The first stage of working with a Swift in a Playground.

5) Swift is an _____ source project, which means its source code is available, and development is done in public.

6) Other programming languages use these to signify the end of a line. Swift does not.

8) A feature of Swift Playgrounds that allows you to look at the result or output of a line of code.

10) The third stage of working with a Swift in a Playground.

12) A Swift _____ is an environment for running code, and exploring Swift.

Down

1) This function is used to output text from Swift.

2) Apple's big, powerful Swift integrated development environment (IDE).

4) The popular phone, made by Apple, that you can write Swift apps for.

7) Check, change, or combine a value with one of these.

9) The second stage of working with a Swift in a Playground.

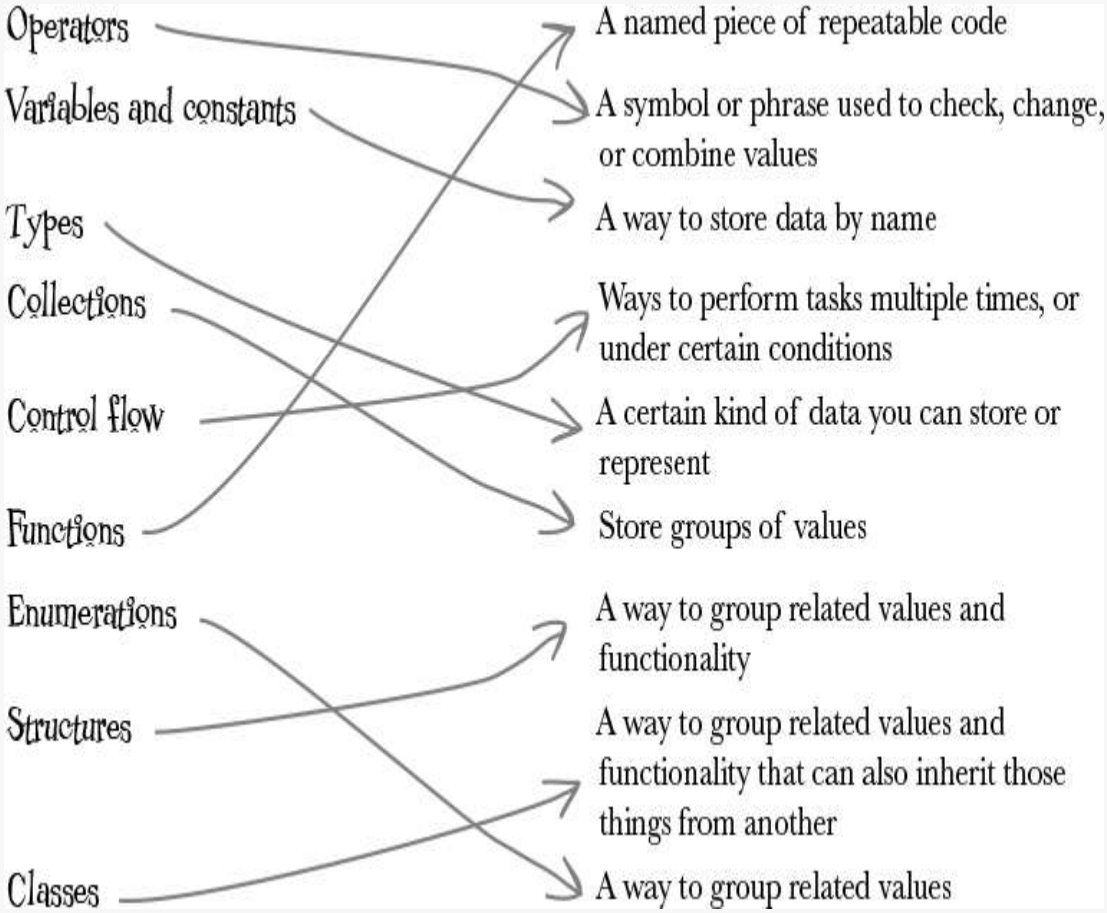
11) A ____ method is the starting point for many programs in other languages, but isn't needed in Swift.

EXERCISE SOLUTION

Arrays start at 0. This means access index 7 of an array will actually access the 8th item of the array.

`ingredients[7]` will access "`Oregano`".

WHO DOES WHAT? SOLUTION



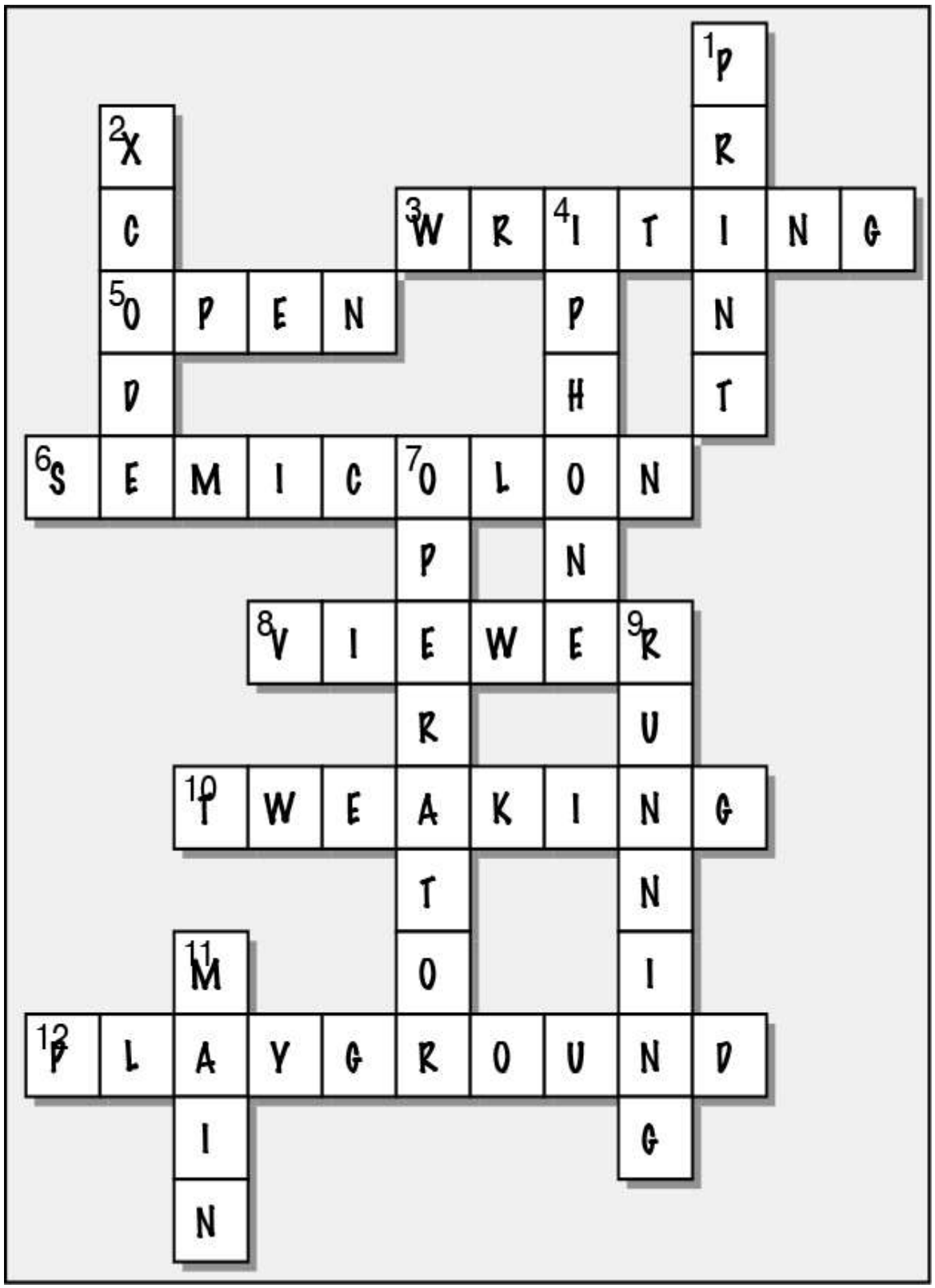
EXERCISE SOLUTION

The last element in the `ingredients` array is "`Capers`".

After you've appended a new ingredient ("`Banana`") to the array, the new last ingredient will be "`Banana`", because `append` adds things to the end of the array.

Exercise Solution

Swiftcross


1 P
2 X
R
C
3 W R 4 I T I N G
5 O P E N P N
D H T
6 S E M I C 7 O L O N
P N
8 V I E W E 9 R
R U
10 T W E A K I N G
T N
11 M O I
12 P L A Y G R O U N D
I G
N

Chapter 2. Swift by name: Swift by Nature

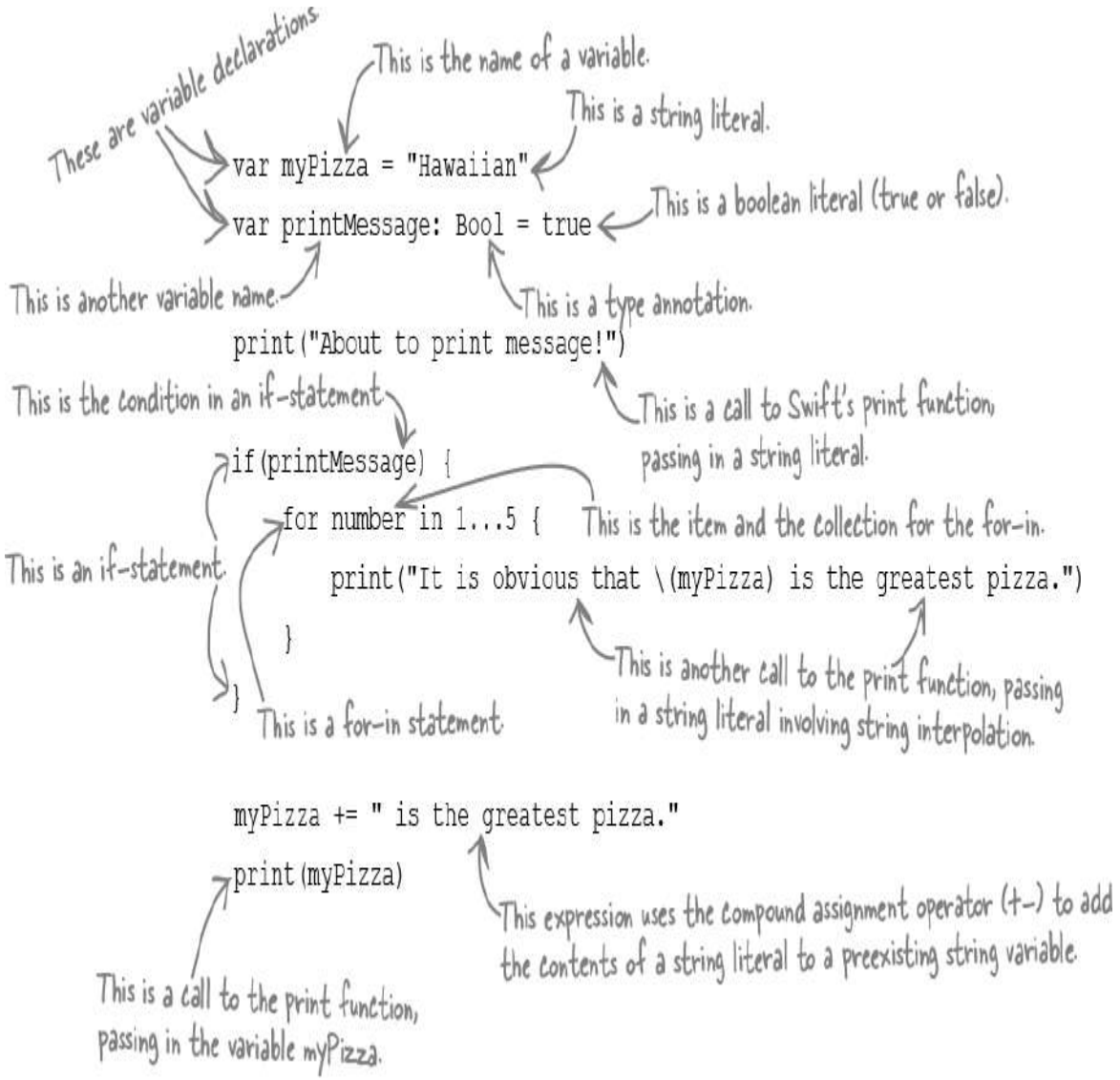
You already know the gist of Swift. But it's time to dive into the building blocks in more detail. You've seen enough Swift to be dangerous, and it's time to put it into practice. You'll use Playgrounds

to write some code, using **statements**, **expressions**, **variables**, and **constants**. The fundamental building blocks of Swift. In this chapter, you'll build the foundation for your future Swift programming career. You'll get to grips with **Swift's type system**, and learn the basics of **using strings to represent text**. Let's get going... and you can see for yourself how swiftly you can write Swift code.

> **NOTE**
>
> The puns will now stop.

Building from the blocks

Every program you write in Swift is *composed* of different elements. Through the chapters of this book, you'll be learning what those elements are, and how to combine them to get Swift to do what you want it to do.


These are variable declarations.
This is the name of a variable.
This is a string literal.
var myPizza = "Hawaiian"
var printMessage: Bool = true
This is a boolean literal (true or false).
This is another variable name.
This is a type annotation.
print("About to print message!")
This is a call to Swift's print function, passing in a string literal.
This is the condition in an if-statement.
if(printMessage) {
for number in 1...5 {
This is the item and the collection for the for-in.
This is an if-statement.
print("It is obvious that \(myPizza) is the greatest pizza.")
}
This is another call to the print function, passing in a string literal involving string interpolation.
}
This is a for-in statement.
myPizza += " is the greatest pizza."
print(myPizza)
This expression uses the compound assignment operator (+=) to add the contents of a string literal to a preexisting string variable.
This is a call to the print function, passing in the variable myPizza.

BULLET POINTS

- There are numerous elements that make up a Swift program. These are the building blocks.
- The most fundamental building blocks are expressions, statements, and declarations.
- A statement defines an action for your Swift program to take.
- A expression returns a results a result, a value.
- A declaration introduces a new name into your Swift program, for example for a variable.
- Variables let you store named data of a certain type.
- Types define what kind of data is stored in something.
- Literals represent literal values (e.g. 5, or true, or “Hello”).

Basic Operators

The first building block we’re going to look at is operators. An operator is a phrase or a symbol that is used to change, check, or combine values that you’re working with in your program.

Operators can be used to do mathematics, do logical operations, assign values, and more. Swift has all the operators that are common building blocks of most programming language, plus a whole bunch that are reasonbly unique to Swift. You’ll learn more about operators as you read more of this book, including how to create your own.

Operators can be *unary*, *binary*, or *ternary*:

This is just a fancy way of saying single target (unary), two target (binary), and three target (ternary).

```
-1
11 + 2
a ? b : c
```

This – is an unary operator. It operates on one value, in this case 1..

This + is a binary operator. It operators on two values, in this case 11 and 2..

This ? and : are components of a ternary operator. It operates on three values, in this case: a, b, and c. You'll learn more about this later.

An operator is a phrase or symbol that lets you change, check, or combine values.

RELAX

It’s easy to feel intimidated if you don’t understand, or know, all of the potential operators and symbols for them.

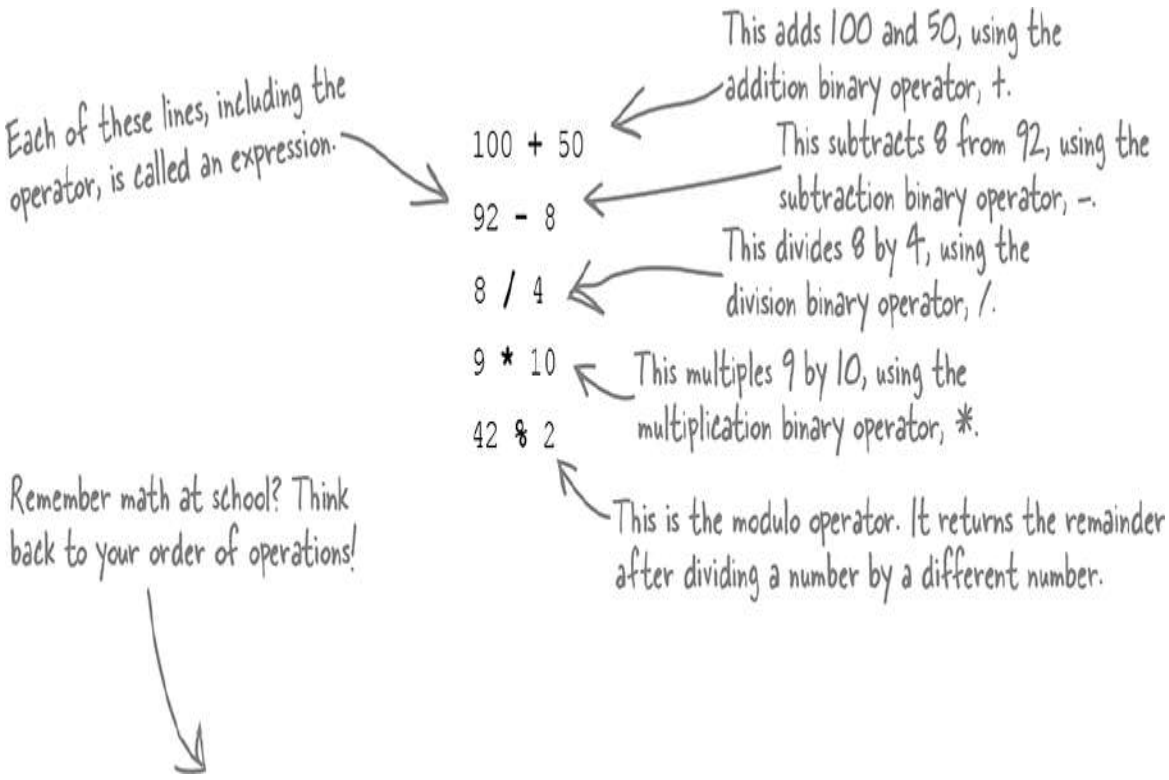
As you increase your knowledge and experience with Swift, many of the operators will become second nature (and many will already be second nature, like the math operators), but many won’t.

It’s totally normal for a programmer to have to look up operators, keywords, and syntax for programming, even if they’ve been programming in that language for years, or decades. It’s totally normal to have to check your memory by looking things up.

Don’t fret if you can’t remember, don’t want to remember, or feel like it’s just not sticking. Totally normal.

Operating swiftly with mathematics

When you’re programming, you’re often doing math. Swift isn’t going to let you down when it comes to doing math. All the classics are there, including ***adding, subtracting, dividing***, ***multiplying,*** and ***modulo***:

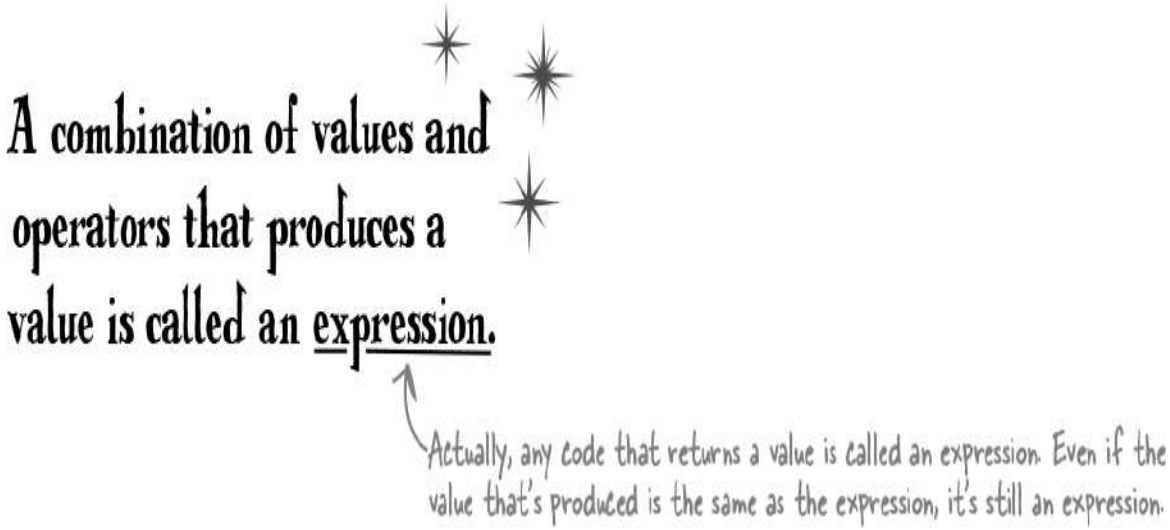


EXERCISE

Create a new Playground and do some math! You can combine operators, just like high school.

Try multiplying something by 42, and then dividing it by 3 and subtracting 4. Try 4 + 5 * 5.

What do you think the result will be? Check if a number is even by using the modulo operator (if there's no remainder from dividing something by 2, then it's an even number).



Expressing yourself

If you input these **expressions** into a Playground and run it, you can see what their value is (what they *evaluate* to):

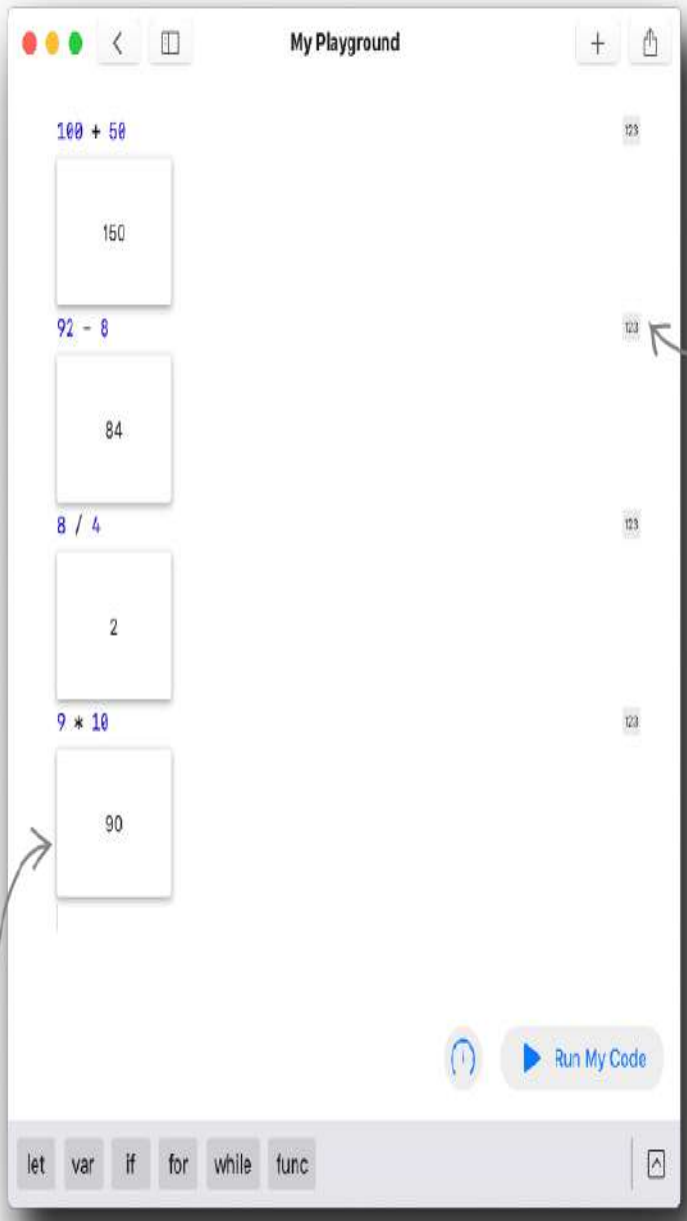
```
100 + 50

92 - 8

8 / 4

9 * 10
```

Do this!



Because each of these expressions has a result, you can add a viewer for them.

The results viewers that you add will appear below each expression that you add them for.

WATCH IT!

You can’t mix and match whitespace styles...

...you have to pick one style and stick with it. So your expressions and operators can either be like `8/4` *or be like* `8 / 4` *but they can’t be* `8 / 4` *or be like* `8/ 4`

BULLET POINTS

- Swift programs are built from statements.
- A simple statement is built from expressions and declarations.
- A expression is code that evaluates to a result.
- A statement does not return a result.
- A declaration is when a new variable, constant, function, structure, class, or enumeration is introduced.
- Operators are phrases or symbols used to change, check, or combine values in Swift.
- Some of the most common operators let you perform mathematical operations.
- Other operators let you assign things, compare things, or heck the result of things.

WHO DOES WHAT?

Solidify your knowledge of the basic operators you can use in Swift expressions, and match the operator and expression on the left to its result on the right..

7 + 3	2
9 - 1	10
10 / 5	20
5 * 4	8
9 % 3	0
-11	-11

If you only use whole numbers in your expression, then the result will also be a whole number.

If you use decimal numbers anywhere in your expression, the result will also be a decimal number.

Say you want to divide `10` by `6` (to which you'd expect the answer to be something resembling `1.666666666666667)...` if you did this:

```
10 / 6
```

You'd end up with a result of `1`, because the result can only be a whole number and it's been rounded down to the closest one.

Whereas, if you ask Swift to use decimal numbers, like this:

```
10.0 / 6.0
```

You'd end up with the more precise answer of `1.666666666666667`, because Swift is working exclusively with decimal numbers.

Swift also has another math operator: ***modulo***. You can use it to figure out how many multiples of a number will fit inside another number.

For example if you wanted to know how many times `2` would fit inside `9`, you'd do this:

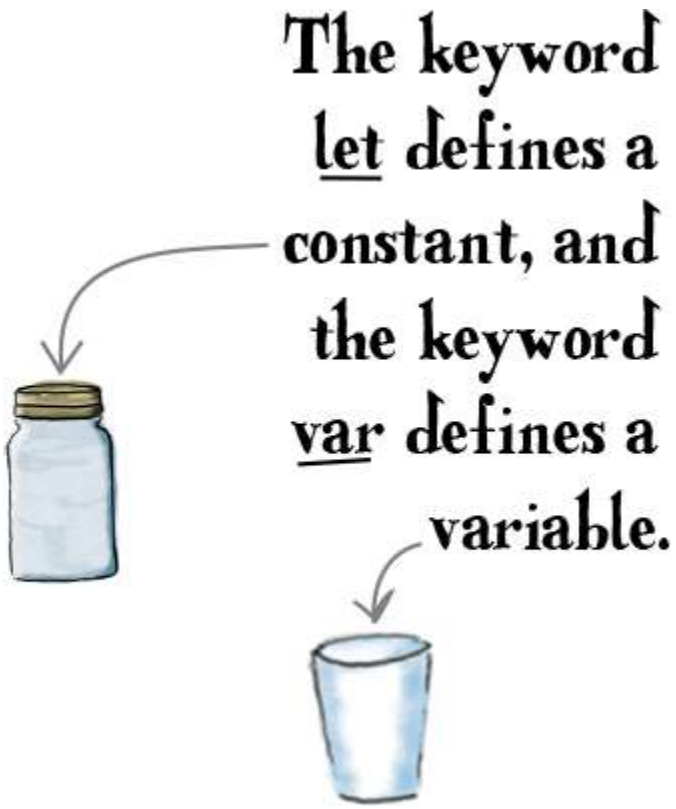
```
9 % 2
```

The answer to this is `1`. You can fit `4` units of `2` inside `9`, before the remainder that's left isn't enough to make another `2`—only `1` remains.

> **NOTE**
>
> **In Swift, whole numbers are more properly known as "integers", and decimal numbers as "doubles" or "floats". Integers, doubles, and floats are known as Types.**

Names and types. Peas in a pod.

Like pets and seasonal beverages, it's much easier to work with data that has a name. **Constants** and **variables** are used to store some data, and refer to it by name. You assign data to them using the **assignment operator**, =. A **variable** is a piece of named data for which *the value can be changed*, while a **constant** is one where *the value can never be changed*. Here are a few examples:



This = is another operator. It's called the assignment operator.

```
let favNumber = 8.7
```

This statement declares a constant named favNumber and assigns the double value of 8.7 to it.

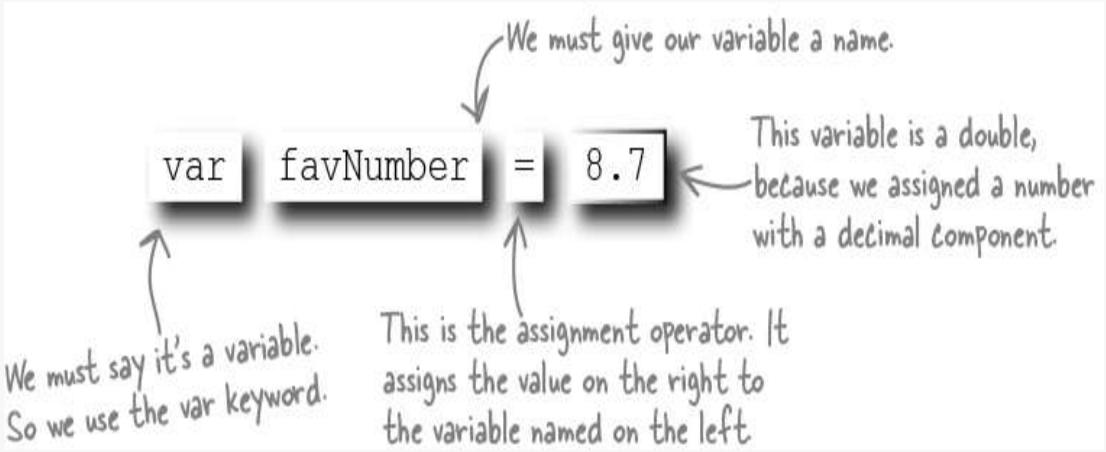
```
var coffeesConsumed = 17
coffeesConsumed = 25
```

This one declares a variable named coffeesConsumed and assigns the integer value of 17 to it.

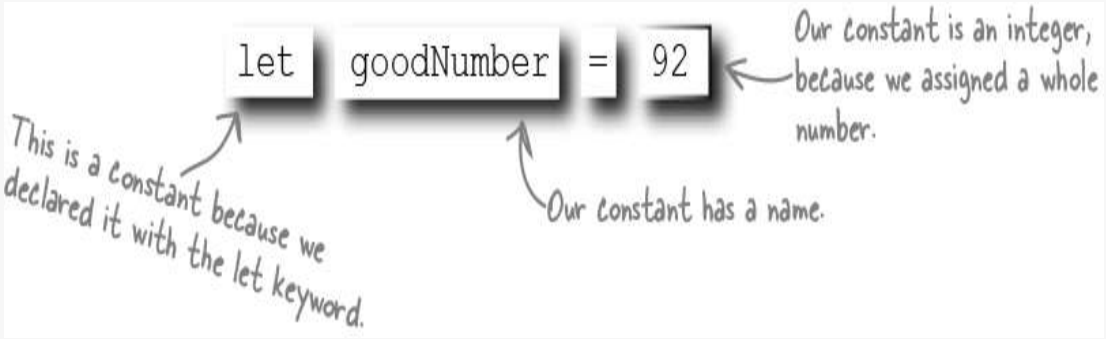
A variable can be updated later. Because it's variable. Get it?

VARIABLES AND CONSTANTS UP CLOSE

Declaring a variable



Declaring a constant



You can't change a constant once it's set. Ever. Not even if you're the boss.

The value of a **variable** can be changed as many times as you need to, but once you've assigned a value to a **constant** you can't change the value. Let's take a little closer look at the ways you can declare variables and constants:

```
let myNumber = 10
```
This defines a constant, named myNumber, and assigns the value 10 to it. Swift figures out that this constant is a whole number, an integer, because we assigned the value as one.

```
let myDouble = 10.5
```
This expressions creates a constant called myDouble storing the decimal number 10.5 in it (a Double).

```
var number = 55
```
This expression declares a variable called number, which we assign the integer 55 to, making the variable an integer.

```
number = 9.7
```
Because number is an integer, we can't assign a double to it (even though it's a variable). This will error.

```
number = 10
```
You can update number to a new integer, though. Because it's a variable.

```
let myConstant = 5
```
This creates a constant, named myConstant, and assigns the integer 5 to it. Making it a constant integer.

```
myConstant = 7
```
Once it has a value it can't be changed. So we can't then do this. This will error.

SHARPEN YOUR PENCIL

Create a new Swift Playground, and write some Swift code that does the following:

- ☐ Create a variable named pizzaSlicesRemaining and set it to 8
- ☐ Create a constant named totalSlices and set it to 8
- ☐ Divide totalSlices by pizzaSlicesRemaining
- ☐ Update the value of pizzaSlicesRemaining to 4
- ☐ Divide totalSlices by pizzaSlicesRemaining again.

RELAX

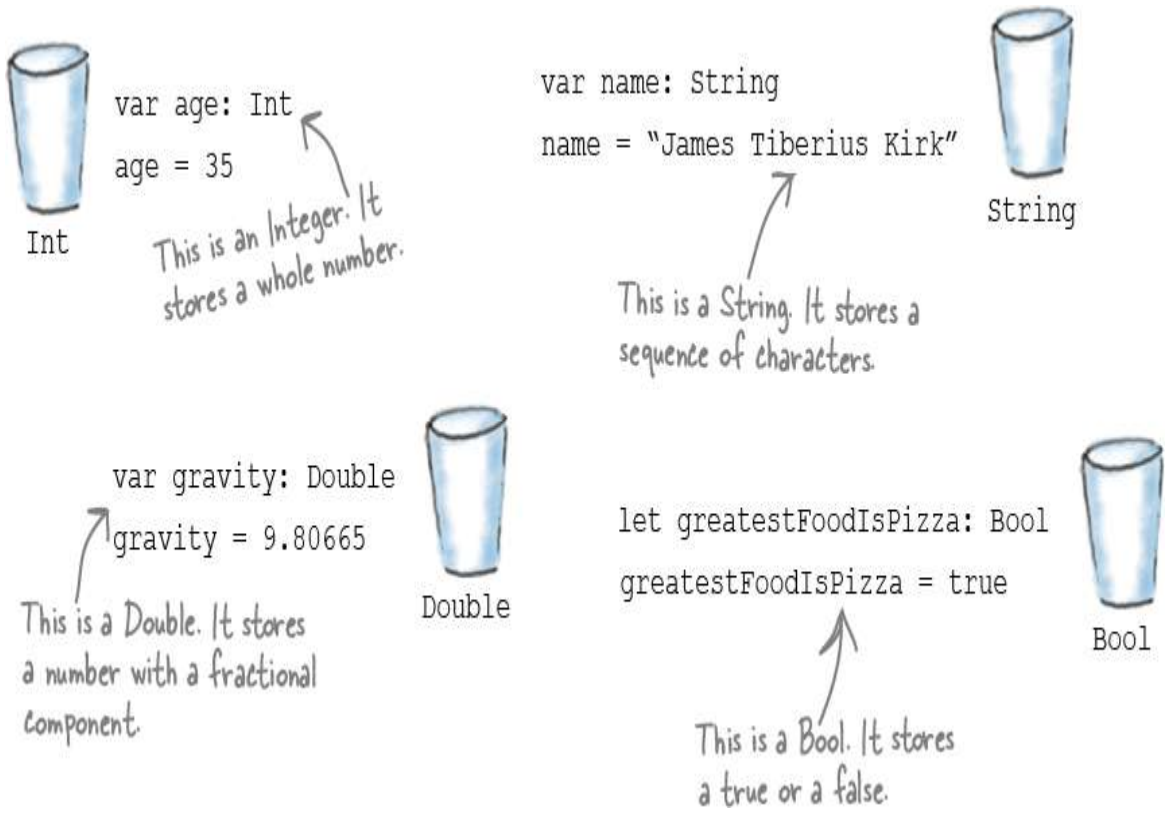
This is a lot to take in. There's no need to be intimidated.

There's a lot of elements to consider, even at this early point in your Swift journey. You'll build on your knowledge, and you'll feel more comfortable as we continue. We promise.

Not all data are numbers

You already know how to create and name both constants and variables when you need to store some that in numerical form (whether or not a decimal poin is involved!) There's more than just numbers though. It's time to introduce one of the biggest stars of the Swift world: the **type system**.

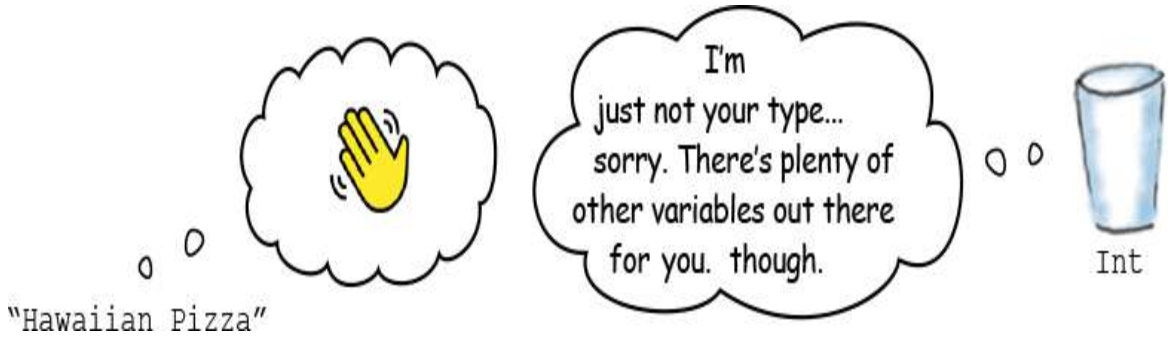
These are all types



Sometimes you want a number, sometimes you want a string, sometimes you want something else entirely. Swift has a **type system**, which lets you tell Swift what kind of data to expect using a **type annotation** or **type inference**.

You've already seen some types: integers, doubles, floats, and strings, and we've been setting the type of our variables and constants using ***type inference***: where we let Swift figure out what type of data something is while it's assigned.

There's a bunch more to learn, though. We'll cover them as we go through this chapter, and the book. Once a variable has its type set, there's no changing it, and there's no assignment some data that's of the wrong type:



EXERCISE

Create a new Playground and declare an Integer variable, assign some data (an integer, naturally) to it, and then, on the next line, attempt to assign a string to it, and see what happens. What error does Swift give you?

While it might be conceptually obvious that variables are designed to have their values change, and constants are not, it pays to look at it in practice. Create a Swift Playground, and declare the following variable, reprenting the name of a pizza shop:

```
var pizzaShopName = “Big Mike’s Pizzeria”
```

If the pizza shop gets acquired, and the new owner (which, naturally, is no longer Big Mike) wants to change the name to “Swift Pizza”, because the pizzaShopName is a variable, you can add something like this to your Playground and there’ll be absolutely no problem:

```
pizzaShopName = “Swift Pizza”
```

However, if you change the first line in your Playground to this:

```
let pizzaShopName = “Big Mike’s Pizzeria”
```

You’ll see that the second line, where we try to change the name, now has an error (as does the first line, suggesting you change it to a variable, to make it mutable).

This is one of the simplest safety features of Swift: if something isn’t meant to change, Swift will tell you about it.

Stringing things along with Types

When you're programming, as we said, you're often doing math. But you're also often working with text. Programming languages represent text using a type called a string. As you saw on the previous page, you can create a string like this:

```
var greeting = "Hi folks, I'm a string! I'm very excited to be here."
```

Or like this:

```
var message: String = "I'm also a String. I'm also excited to be
there."
```

When you include a predefined value that happens to be a string in your code like we just did, it's called a **string literal**. A string literal is, quite literally, a sequence of characters surrounded by double quotation ("") marks.

If you want to create an empty variable that stores a string, you can do that too, like this:

```
var positiveMessage = ""
```

In this case, positiveMessage is a String because the data "" is a string (however short), and type inference makes the variable a string.

Or like this:

```
var negativeMessage: String
```

negativeMessage is a String because a String type annotation is provided.

You can then assign values to your strings:

```
positiveMessage = "Live long and prosper"
negativeMessage = "You bring dishonour to your house."
```

WATCH IT!

Types are, by design, inflexible.

You cannot do this:

```
var pizzaTopping
pizzaTopping = “Oregano”
```

This is because, when we declared the variable `pizzaTopping` *we didn’t supply any data for Swift to perform Type Inference with, and we also didn’t supply a Type Annotation explicitly telling Swift what Type it would be.*

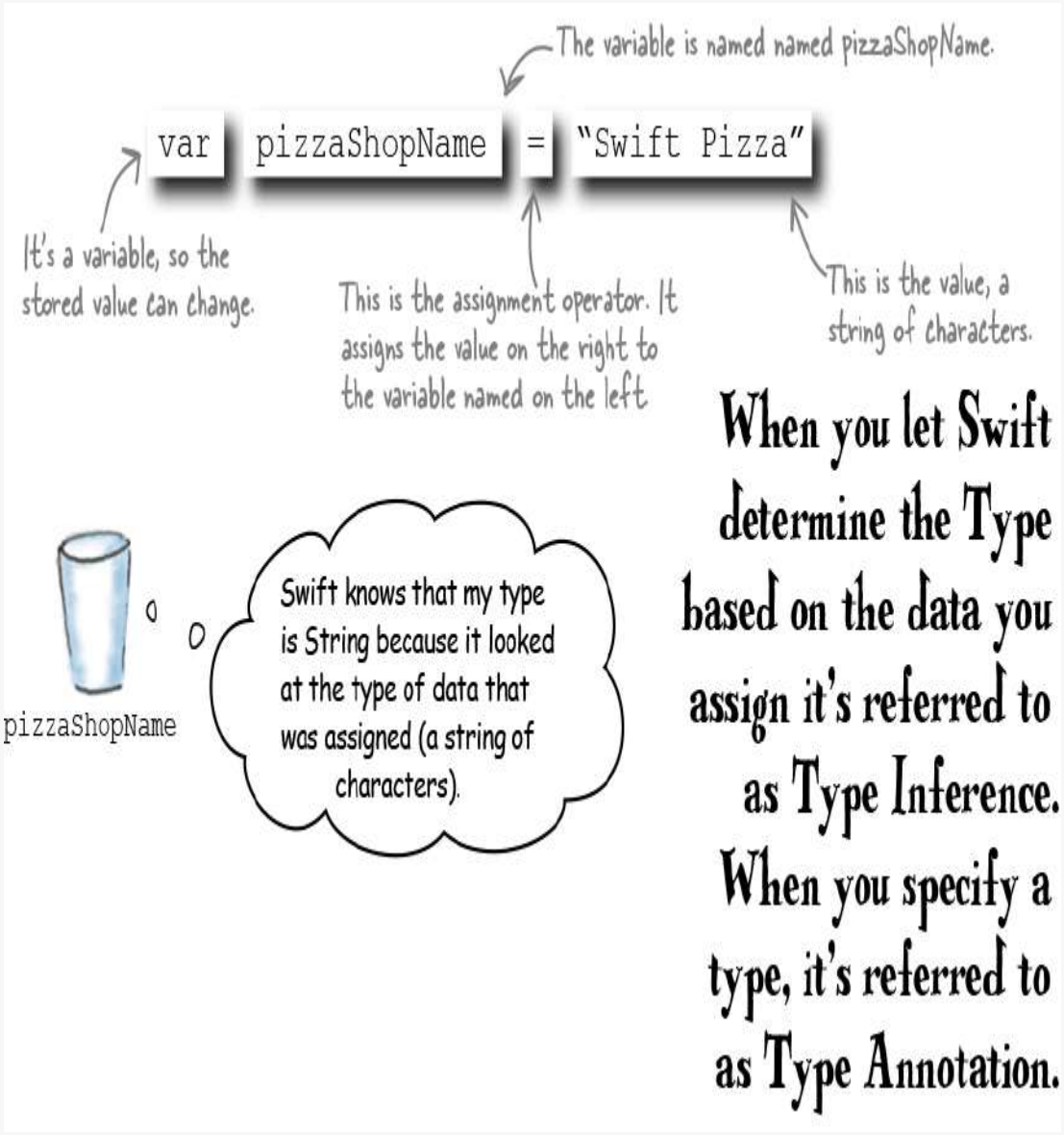
So, while it’s normally perfectly fine to declare a variable and then assign some data to it (or change the data) at a later point—that’s the entire point of a variable—in this case, because we declared the `pizzaTopping` *variable without a type, and without any data to infer a type from, it’s not safe, so it can’t be done.*

If we want to declare a variable in advance without supplying some data, we must supply a Type Annotation, like this:

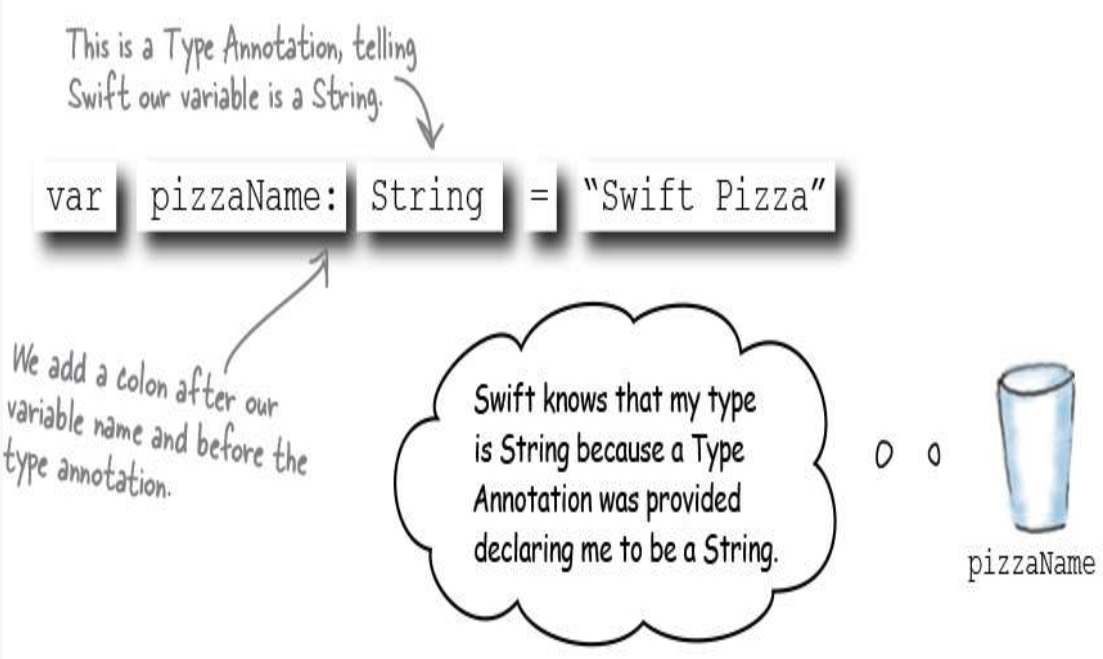
```
var pizzaTopping: String
```

ANATOMY OF A TYPE

A String with Type Inference

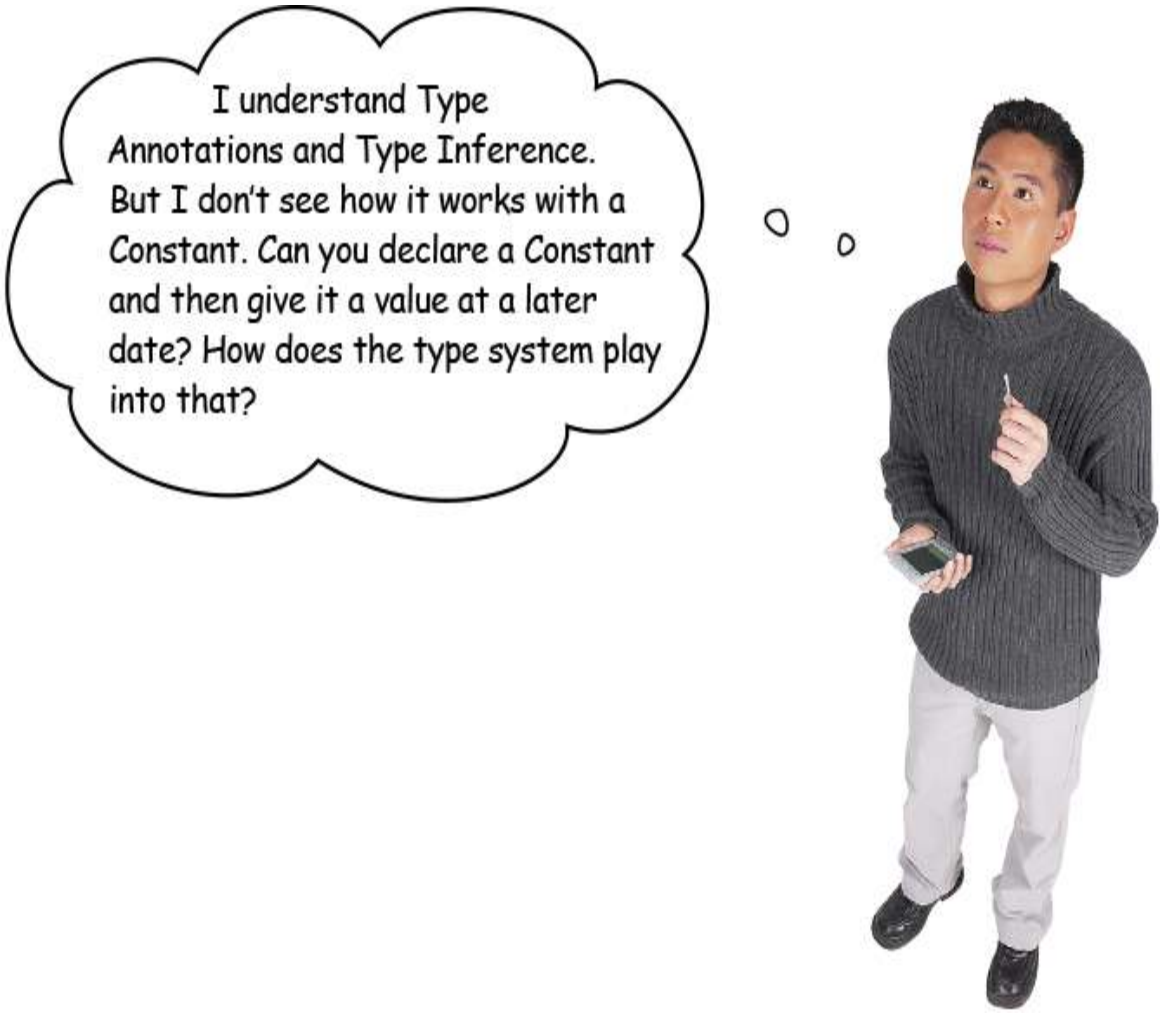


A String with Type Annotation



BULLET POINTS

- String stores strings of characters. There could be any number of characters in a string, including 0.
- Int stores a whole round number. No decimal point. They could be negative though.
- Double stores numbers that have a decimal point. They could also be a negative.
- Bool stores true or false. Nothing else.



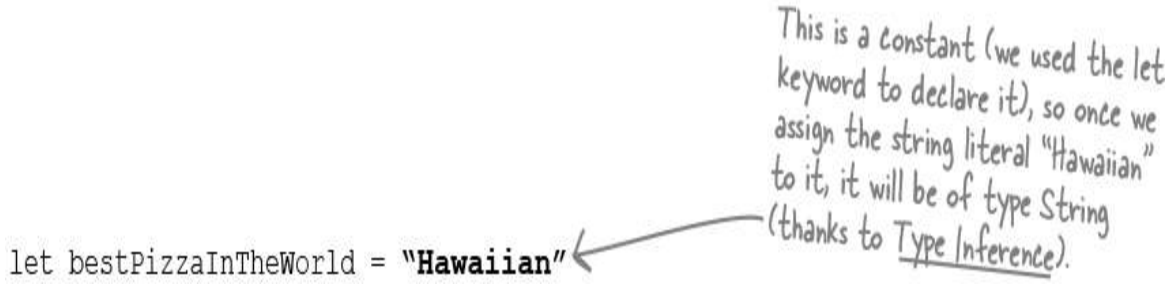
Constants and Variables need a Type Annotation, or a value that is assigned at declaration. Or both.

When you declare a variable or constant, you can **either supply a Type Annotation, or immediately assign a value**.

If you assign a value, that's all you need to do, the type system will use type inference and you're done.

If you don't want to assign a value at declaration, that's fine, but you must supply a type annotation so that Swift knows what type of data to expect when some data is assigned.

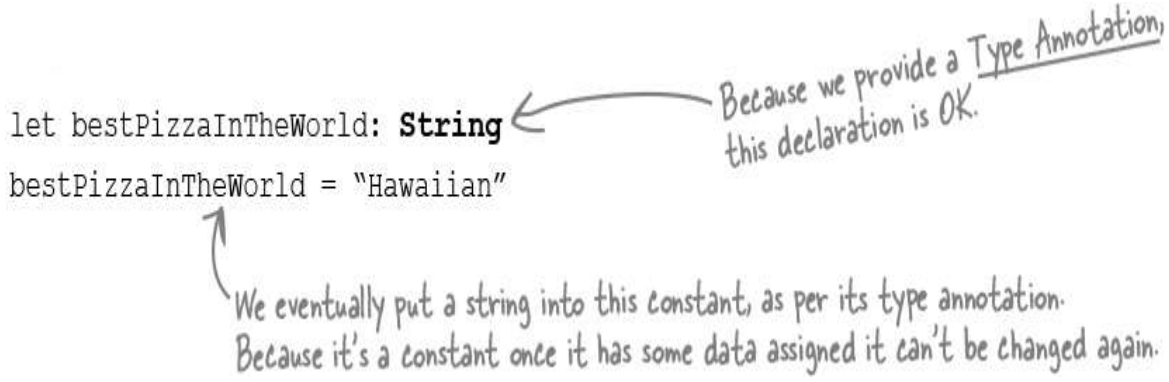
For example, this is fine because we assigned some data to this constant at the time of declaration:



But if we do not assign any data at the time the constant or variable is declared, there will be an error:

```
let bestPizzaInTheWorld
bestPizzaInTheWorld = “Hawaiian”
```

We must supply a type annotation if we want to do this, like this:



Who am I?

A bunch of Swift **Types**, in full costume, are playing a party game, “Who am I?” They’ll give you a clue—you try to guess what their name is based based on what they say about themselves.

Assume they always tell the truth about themselves. Draw an arrow from each sentence to the name of one attendee. We’ve already guessed one of them for you, and you can check your answers at the end of the chapter.

If you find this challenging, it’s totally OK to check the answers instead.

	Tonight’s attendees:
I exclusively focus on whether things are true or false. I don’t care about anything else.	**String**
I just love characters and prefer to focus on one character at a time.	**Bool**
I am interested in working with as many characters as I can, all at once.	**Int**
I love whole numbers. Positive and negative. I just don’t do fractions.	**Double**
I’m in love with fractional components. I’m a big fan of numbers that have them. That’s all I like, really.	**Character**

NOTE

You can only use the += operator to add to a string. You cannot use -= to remove from a string.

Modifying a string after you created it is easy. It, mostly, just works like it does for numbers.

You can add something to a string. Let’s say you were storing a nice motivational speech:

```
  var speech = “Our mission is to seek out new
lifeforms, and new civilizations.”
```

And then, later on, you wanted to add some more:

speech +=

“To boldly go where nobody has gone before.”

> **NOTE**
>
> This += is an operator. It’s called the a compound assignment operator. It means take speech, and store speech plus the following thing in speech.

The `speech` variable would now contain:

```
“Our mission is to seek out new lifeforms, and new
civilizations. To boldly go where nobody has gone before”
```

EXERCISE

Create a new Swift Playground, and code the following:

- A String variable named favouriteQuote
- Assign a snippet of your favourite quote to the variable
- Add the string “by” and then the author of the quote to the favouriteQuote variable
- Print the favouriteQuote variable

String interpolation

You’d be forgiven for thinking string interpolation was something from science-fiction, but it’s not, and it’s very useful. **String interpolation** lets you construct a string from a mix of values—constants, variables, string literals, and expressions—by including their values inside a new string literal.

Let’s say you had a number that represented the speed of your hypothetical spaceship, measured in warp factors (just like on a certain popular science-fiction television show):

```
var warpSpeed = 9.9
```

And you also had the name of the hypothetical spaceship, stored as a string constant:

```
let shipName = “USS Enterprise”
```

And your ship was en route to some far-off planet, and you had that stored as a variable:

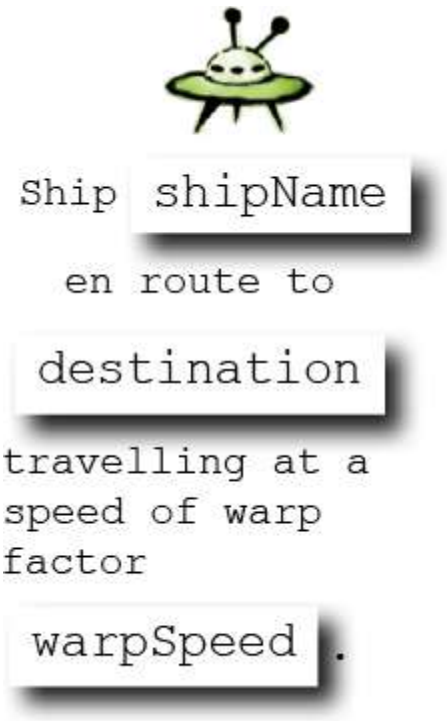
```
var destination = “Ceti Alpha V”
```

You could use string interpolation to construct a brand new string, incorporating all this useful information:

```
var status = “Ship \(shipName) en route to \(destination),
travelling at a speed of warp factor \(warpSpeed).”
```

If you then printed this new status string variable, the result would be:

```
Ship USS Enterprise en route to Ceti Alpha V, travelling
at a speed of warp factor 9.9.
```



The placeholder is replaced wiht the actual value of the constants, variables, string literals, and expressions being named.

You might have noticed us using string interpolation when calling Swift’s print function, to display the values of variables and constants as part of the printed output.

EXERCISE

Create a new Swift Playground, and add the following code:

```
var name = “Head First Reader”
var timeLearning = “3 days”
var goal = “make an app for my kids”
var platform = “iPad”
```

Use String interpolation to print a string that says “Hello, I’m Head First Reader, and I’ve been learning Swift for 3 days. My goal is to make an app for my kids. I’m particularly interested in the iPad platform.” Use the variables and string interpolation to customise the printed string with your own details.

Literal values are any value that appears directly in your code.

For example, if you define a variable, named `peopleComingToEatPizza`, and you assign the value of 8 to it (because that's how many people you have coming over to eat pizza) then the value that appears directly in your source code, which is 8, is an integer literal:

```
var peopleComingToEatPizza = 8
```

If you define a variable to represent how much of your delicious pizza pie is remaining, by percentage, and assign the value `3.14159` to it, then you've assigned a floating-point literal to your variable:

```
var pieRemaining = 3.14159
```

And if you described a pizza using words, and stored it in a variable, you've created a String variable thanks to type inference because you assigned a string literal:

```
var pizzaDescription =
      "A delicious mix of
           pineapple and ham."
```

There are no dumb Questions

Q: **Where are the semicolons? I thought programming was meant to be full of semicolons!**

A: Swift doesn't end lines with semicolons. If it makes you feel more comfortable, you can do it anyway. It's still valid, it's just not necessary!.

Q: **I thought Swift had something called "protocols" and classes? When are we learning those? What about classes?**

A: Swift does have something called "protocols", and we promise that we'll get to it. We'll also get to classes very soon. Swift has more than one way to structure a program, and both classes and protocols offer useful paths forward. Be patient, and we'll get there.

Q: **How do I run my code on an iPhone or iPad? I bought this book so I could learn to make iOS apps and get filthy rich.**

A: We'll explain everything you need to know to use your Swift knowledge to make iOS apps much, much later in the book. We

make no promises about getting rich, but we do promise that you'll know how to build iOS apps by the end. Again, be patient.

Q: **Why would I ever use a constant when I could just make everything a variable in case it needs to change?**

A: Swift places a much greater emphasis on using constants than other programming languages do. It can help, a lot, with making your program safer and more stable by only using variables for values that you expect to change. Additionally, by telling the Swift compiler that your values with be constants, the compiler can make our programs faster by performing certain optimizations for us.

Q: **Why can't I change the type of a value after I've assigned it? Other languages can do this.**

A: Swift is a strongly typed language. It puts a lot of emphasis on the way type system works, and encourages you to learn it. You cannot change the type of a variable after you've created it. You can create a new variable and cast the type to something new if you need to.

Q: **So are variables and constants also types?**

A: No, variables and constants are named locations to store data. The data in a variable or constant has a type, so the variable or constant has a type, but it in itself is not a type.

Q: **What if I need to store some data that Swift doesn't have a type for?**

A: Great question. We'll be looking at ways you can create your own types later on in the book.

Q: **What about the Collection Types? Can I make my own Collection Type too?**

A: Yes, we'll get back to this in a chapter or three, as it requires some advance Swift features. But the answer is yes: you can make your own Collection types that are just as capable as the provided three (arrays, sets, dictionaries).

Q: **What do I do if I need to create a variable, but don't know what type to set it to in advance?**

A: Another great question! You might have noticed that in our code so far, sometimes we specify a type in the expression that creates a variable, and sometimes we don't. Swift's type system is able to infer types, or use a provided type annotation.

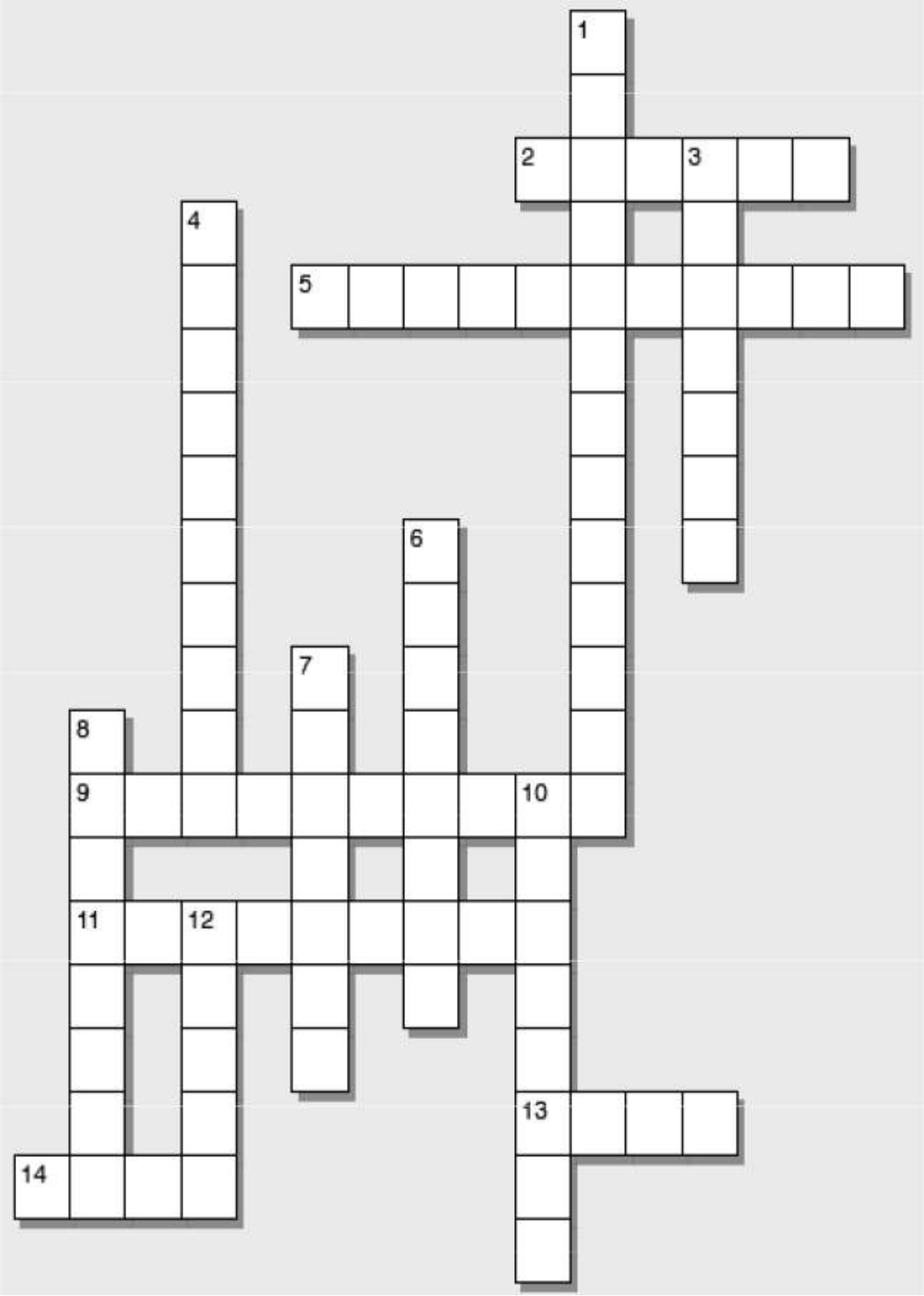
BULLET POINTS

- Operators are a phrase or a symbol that is used to change, check, or combine values that you're working with in your program.
- Variables and constants let you store data, and refer to it by name.
- Variables and constant have a type (like integer, or string). Swift will assign a type for you, based on what you store in the variable or constant at creation, or you can explicitly specify a type.
- Variables can have their value, but not their type, changed at any time.
- Constants can never changed value or type.
- Swift programs are comprised of expressions, and statements, and declarations.

Swiftcross

It's time to test your brain. Gently.

This is just a normal crossword, but all the solutions are from concepts we covered in this chapter. How much did you pay attention?



Across

2) A type that stores words, characters, or sentences.

5) Something that introduces a new name into a program.

9) A type _____ is used to mark a type on a variable.

11) Type _____ is what we call it when Swift figures out the type of something based on what the data looks like.

13) What kind of data is stored.

14)

Down

1) String _____ is when a string is placed within another string.

3) The type for a whole number.

4) A statement that returns a result (a value).

6) A named piece of data where you cannot modify the data after it’s created.

7) A value typed in your code.

8) Mutable stored, named, data.

10) A phrase or symbol that is used to change, check, or combine values.

12) The type for a number with a decimal place.

a. Building from the blocks
b. Basic Operators
c. Operating swiftly with mathematics
d. Expressing yourself
e. Names and types. Peas in a pod.
f. Not all data are numbers
    i. These are all types
g. Stringing things along with Types
h. String interpolation
i. There are no dumb Questions
j. Swiftcross